Guía de defensa del proyecto

Sistema de Asistencia Docente

Todo el proyecto: dónde está cada cosa, cómo explicarla y cómo cambiarla

Dato	Detalle
Trabajo	Trabajo Intercátedra · Práctica Profesionalizante 3 · Grupo 3 (IESETyFP)
Alcance	**Todo el sistema**: Personal (Pantalla 1), Asistencia (Pantalla 2), Licencias (Pantalla 3) y Administración / Reportes (Pantalla 4)
Base de esta guía	Repositorio del zip actual (Proyecto_Sistema_Asistencia-main), código leído archivo por archivo, más el Documento Master de Requerimientos y Evaluación. Última corrida de pruebas: **516 tests OK** (personal 33 · licencias 136 · reportes 97 · asistencias 250)
Defensa	Viernes 13/11/2026 · evaluación individual

Cómo usar esta guía

- **Si te preguntan «¿dónde está…?»**, andá a la sección 1 (hoja rápida): pregunta, archivo y qué decir, de los cuatro módulos.
- **Si te preguntan por un módulo**, las secciones 3 a 6 explican cada uno: pantallas, modelos, reglas, archivos y una frase de 30 segundos.
- **Si el docente compara contra la rúbrica o sus observaciones** (roles, zona horaria, marcas dobles, mensajes), andá a la sección 7: dice qué se cumple, qué es distinto y cómo explicarlo con honestidad.
- **Si te piden «cambiame esto en vivo»**, la sección 12 tiene recetas paso a paso con los archivos exactos.
- **Si te hacen una pregunta teórica o de reglas de negocio**, la sección 11 trae preguntas con respuesta corta y el archivo donde se ve.
- **Si te marcan un defecto**, la sección 14 lista los puntos débiles verificados en el código y cómo responderlos.

Contenido

1. Hoja rápida: «si te preguntan X, abrí Y»
2. Visión general del sistema
3. Personal (Pantalla 1)
4. Asistencia (Pantalla 2)
5. Licencias (Pantalla 3)
6. Administración / Reportes (Pantalla 4)
7. Rúbrica de la cátedra y observaciones del docente
8. Cómo se conectan los módulos (recorridos de punta a punta)
9. Roles y permisos: quién puede qué
10. Modelo de datos
11. Banco de preguntas y respuestas
12. Recetas: si te piden cambiar algo en vivo
13. Comandos útiles
14. Puntos débiles conocidos y cómo responderlos
15. Mapa de archivos
16. Guion de demo y checklist de la víspera

1. Hoja rápida: «si te preguntan X, abrí Y»

Los números de línea no se citan porque se mueven cuando alguien edita el archivo: buscá por el nombre de la función.

Si te preguntan por…	Abrí	Qué decir en una frase
Personal		
Legajo y su clave	`personal/models.py` → `Persona`	El DNI (texto, 7 a 10 dígitos) es la clave primaria y no se edita; «baja» es `activa = False`.
Designación (centro del modelo)	`personal/models.py` → `Designacion`	Persona + cargo + materia + situación de revista + vigencia; todas las FK son `PROTECT`.
Historial automático	`Designacion.save()` → `HistorialDesignacion`	Compara 28 campos antes y después; si cambió alguno deja ALTA, BAJA, CAMBIO_SITUACION, TOMA_POSESION o MODIFICACION.
Suplencias y plazo indeterminado	`personal/models.py` → `Designacion.clean`; `forms.py`	Una suplencia necesita fecha de cese o plazo indeterminado; `reemplaza` apunta al titular y debe estar vigente.
Horas por día	`personal/forms.py` → `DesignacionForm.save`	Horas cátedra de lunes a viernes; 0 borra el horario del día. Alimentan la planilla rápida y el reporte mensual.
Asistente de 5 pasos	`personal/templates/personal/nueva_designacion.html`	El mismo formulario mostrado por pasos con JavaScript: docente, cargo, horas, normativa y vigencia.
Importación CSV	`personal/views.py` → `_procesar_csv`, `_importar_fila_personal`	Máx. 5 MB, UTF-8, columna `dni` obligatoria; cada fila en su transacción; las rechazadas quedan en `ImportacionPersonalError`.
Designaciones vencidas	`cerrar_designaciones_vencidas` (comando)	Da de baja las activas con «hasta» pasado y deja historial BAJA; pensado para correr a diario.
Catálogos	`personal/views.py` → `catalogos_personal`	Cargos, carreras y espacios se desactivan; los tipos de licencia se borran de verdad si no están en uso.
Asistencia		
Dónde viven las reglas	`asistencias/services/__init__.py`	Las reglas van en servicios, cada escritura en `transaction.atomic` y los errores esperados como `ValidationError` o `PermissionDenied`.
Guardar una asistencia	`services/registro.py` → `guardar_asistencia`	Comprobaciones en orden fijo: permiso, sesión, período, jornada, día hábil, designación, marca y recién ahí guarda y deja el evento.
Planilla rápida	`views/planilla.py`; `guardar_planilla`	Una fila por designación vigente que propone Presente o la licencia; es todo o nada.
Marcas P / I / LIC / LT / PR	`models/marcas.py`; migraciones 0003 y 0004	Catálogo cerrado; cada marca define si exige horas, observación o licencia.
Estados de un registro	`models/registro.py`	BORRADOR → VALIDADO; y justificación NO_APLICA, INJUSTIFICADA, EN_TRAMITE o JUSTIFICADA.

Si te preguntan por…	Abrí	Qué decir en una frase
Validar	`services/recepcion.py` → `validar_asistencia`	Secretaría o más; editar un validado exige motivo y lo devuelve a borrador.
Formulario en papel	`recepcionar_formulario`; `TRANSICIONES_JUSTIFICACION`	Injustificada pasa a en trámite o justificada; justificada es estado final.
Cierre de jornada y de mes (RN-06)	`services/cierres.py`	El mes no cierra con borradores; reabrir exige motivo de 10 caracteres o más y permiso especial.
Máximo 2 operadores	`services/sesiones.py` → `MAX_OPERADORES`	Sesión de edición con token y vencimiento a los 15 minutos; se bloquea una fila con `select_for_update`.
Feriados y fines de semana	`services/reglas.py` → `motivo_dia_no_laborable`; `services/calendario.py`	Hábil es lunes a viernes sin feriado cargado; no se toca el calendario de un mes cerrado.
Marcado automático por licencia	`services/licencia_marcado.py` → `marcar_licencia`	Al guardar una licencia deja LIC o LT en cada día hábil con las horas del horario.
Autocompletado de la carga	`services/agenda.py`; `views/registro.py` → `datos_designacion`	Devuelve horas del día y licencias vigentes en JSON para no tipear.
Historial y CSV	`views/registro.py` → `historial`, `exportar_csv`	Filtros combinables; el CSV corta en 5000 filas y lo avisa.
Licencias		
Validaciones de una licencia	`licencias/models.py` → `LicenciaOtorgada.clean`	Orden: fechas, artículo contra situación de revista, superposición y tope anual.
Tope anual (RN-08)	`clean` + `services.dias_usados_en_anio`	Días hábiles del año por DNI y artículo; una resolución de excepción desactiva el tope.
Días hábiles	`licencias/dias.py`	Única fuente de verdad del conteo: lunes a viernes sin `DiaNoLaborable`.
IL-01 / IL-02	`licencias/clasificacion.py`	Un solo diccionario genera los `choices` y la función; una categoría desconocida lanza error.
Disposiciones (RF-16, RN-10, RN-11)	`licencias/services.py` → `emitir_disposiciones_mes`	Solo con N° de formulario, en orden alfabético y con numeración correlativa por oficina y año.
Resoluciones PDF	`models.py` → `validar_resolucion_pdf`; `views.descargar_resolucion`	PDF de hasta 5 MB con cabecera `%PDF-`; se descarga por una vista con permiso, no por URL pública.
Reporte mensual	`licencias/views.py` → `reporte_mensual_licencias`	Horas afectadas por licencia según el horario, sin feriados, más las inasistencias I y PR del registro diario.
Asistente de artículo (RF-15)	`services.articulos_para_asistente`; `forms.SelectDesignacion`	Entrega planilla, tope y plazo de cada artículo y filtra por situación de revista.
Administración / Reportes		
Consultas unificadas	`reportes/selectors.py` → `buscar_consulta`	Licencias más ausencias con los mismos filtros; las fechas filtran por superposición de períodos.
CSV seguro	`reportes/exportaciones.py` → `_texto_seguro`	UTF-8 con BOM para Excel y apóstrofo delante de textos que empiezan con = + - @.

Si te preguntan por…	Abrí	Qué decir en una frase
Alertas por tope de días	`selectors.calcular_alertas`	Suma días por docente y artículo con la misma regla que Licencias; umbral por defecto 80 %.
Alertas de vencimiento	`calcular_designaciones_por_vencer`, `calcular_licencias_por_vencer`	Ventana de 15 a 90 días; las vencidas sin renovar aparecen arriba.
Roles y permisos	`reportes/roles.py`	Tres grupos acumulativos: Bedelía < Secretaría < Subsecretaría.
De dónde salen los roles	`reportes/signals.py` → `crear_roles`; `sincronizar_roles`	Se crean en `post_migrate`; el comando los restablece a los valores por defecto.
Usuarios	`reportes/views.py`; `forms.UsuarioForm`	Un solo rol por usuario; nadie se quita su propio rol; un superusuario solo lo edita otro.
Bitácora	`reportes/signals.py`, `auditoria.py`, `views.bitacora`	Tabla única `EventoAsistencia` con usuario, acción y fotos antes y después.
Quién hizo el cambio	`reportes/middleware.py`, `usuario_actual.py`	El middleware guarda el usuario en una variable de contexto que leen las señales.
Bitácora inalterable	`signals.py` → `impedir_edicion_de_eventos`	Lanza `BitacoraInalterable` al editar o borrar; las FK hacia asistencias son `PROTECT`.
Parámetros	`reportes/models.py` → `ParametroSistema`	Tabla de una sola fila (pk 1) con oficina y ciclo lectivo.
Transversal		
Fecha de hoy y zona horaria	`licencias/dias.py` → `fecha_de_hoy`; `settings.TIME_ZONE`	Una única fuente de «hoy» en Buenos Aires; no se usa `datetime.now()` en ningún lado.
Protección de pantallas	`asistencias/views/_comun.py` → `vista_protegida`; decoradores en las otras apps	`login_required` + `permission_required(raise_exception=True)`: sin permiso, 403.
Redirecciones seguras	`_comun.py` → `_url_segura`	Solo acepta rutas internas en `volver`, `desde` y `next` (evita open redirect).
Cómo se prueban	`*/tests.py`, `licencias/tests/`	`python manage.py test` — 516 tests en la última corrida.

2. Visión general del sistema

2.1 Qué es

Aplicación web hecha en **Django** para gestionar el personal docente de un instituto: legajos y designaciones, licencias otorgadas con sus topes y disposiciones, y la asistencia diaria con cierres de jornada y de mes. Es una versión simplificada del ERS del trabajo intercátedra. Tiene tres roles de usuario y una bitácora de auditoría de todo lo que se cambia.

> **Nota: Idea central para la defensa:** los docentes son datos (legajos y designaciones), no usuarios del sistema. Quienes entran son personal administrativo (Bedelía, Secretaría, Subsecretaría) que carga y controla la asistencia. Más abajo (sección 7) está cómo explicar esto frente a la rúbrica, que habla de roles «Administrador» y «Docente».

2.2 Tecnología

Tema	Cómo está resuelto	Dónde
Framework	Django (requirements pide `Django>=6.0.6` y Python 3.14.4 o superior)	`requirements.txt`
Base de datos	MariaDB/MySQL con `mysqlclient`; credenciales por variables de entorno `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT` (plantilla `example.env`, puerto por defecto 3307)	`config/settings.py`
Scripts SQL	Uno crea la base (utf8mb4) y el usuario; otro carga datos de prueba ficticios y 9 usuarios (3 por rol)	raíz del proyecto
Idioma y hora	`LANGUAGE_CODE = 'es-ar'`, `TIME_ZONE = America/Argentina/Buenos_Aires`, `USE_TZ = True`	`settings.py`
Login	Vistas de `django.contrib.auth` en `/accounts/`; al entrar va a `asistencia:registro_diario`	`config/urls.py`, `LOGIN_REDIRECT_URL`
Interfaz	Plantillas HTML con herencia desde `base.html` (menú lateral por permisos, mensajes), Bootstrap 5.3.3 por CDN y CSS propio	`templates/`, `static/css/`
Archivos subidos	Resoluciones PDF en `media/`; sin `MEDIA_URL` a propósito: se bajan por una vista con permiso	`settings.py`, `licencias/views.py`
Pruebas	`python manage.py test` (516 tests). El requirements también lista pytest, pero no hay configuración	`*/tests.py`, `licencias/tests/`

2.3 Cómo viaja una petición

- El navegador pide una URL, por ejemplo `/asistencia/planilla/`. `config/urls.py` la delega a la app por prefijo.
- Antes de llegar a la vista pasan los middlewares: sesión, CSRF, autenticación y **`UsuarioActualMiddleware`** (de Reportes).
- La vista tiene `@login_required` y `@permission_required(..., raise_exception=True)`: sin permiso responde **403**.
- Un **formulario** valida los datos de entrada; la vista llama a un **servicio** (escrituras) o a un **selector** (lecturas).
- El servicio verifica permiso, sesión de edición, período y jornada, guarda dentro de `transaction.atomic` y deja un evento de auditoría.

- La vista atrapa `ValidationError` y `PermissionDenied`, los muestra con `messages.error` y renderiza la plantilla.

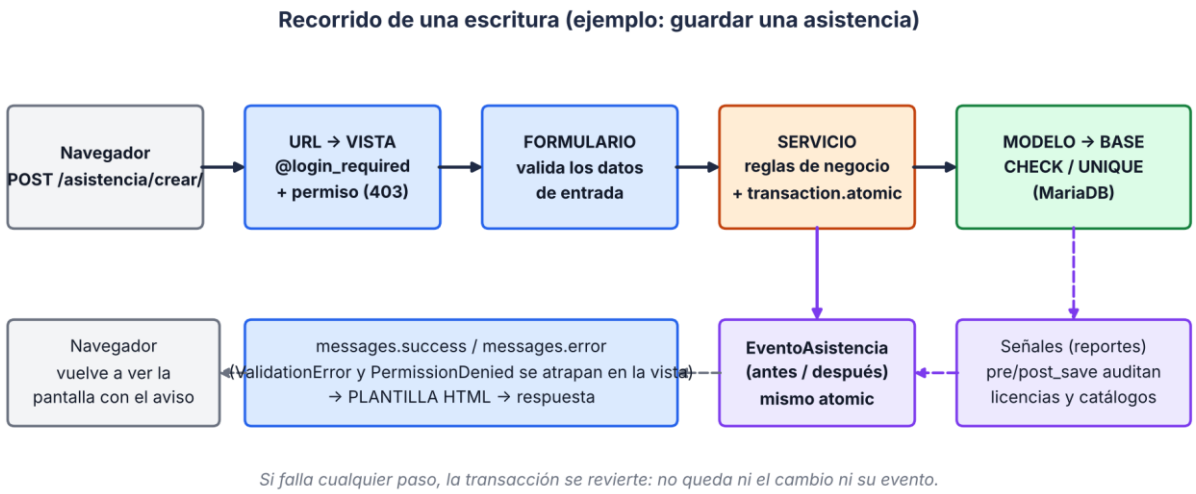


Figura 1. Recorrido de una escritura, de la vista a la base y a la bitácora.

2.4 Las cuatro apps

App	Responsabilidad	Modelos principales	Archivos clave
`personal`	Legajos, designaciones y catálogos; importación CSV; página de inicio	Persona, Designacion, Cargo, Carrera, EspacioCurricular, Horario, HistorialDesignacion, ImportacionPersonalError, catálogos auxiliares	`models.py`, `views.py`, `forms.py`
`licencias`	Licencias otorgadas, topes anuales, disposiciones, reporte mensual IL-01/IL-02, resoluciones PDF	LicenciaArticulo, LicenciaOtorgada, ResolucionAdjunta, RegistroBitacora (histórico)	`models.py`, `services.py`, `dias.py`, `clasificacion.py`
`asistencias`	Registro diario, planilla rápida, validación, recepción de formularios, cierres, calendario, sesiones de edición, bitácora	RegistroAsistencia, MarcaAsistencia, CierreDiario, CierreMensual, EventoAsistencia, SesionEdicion, ControlConcurrencia, DiaNoLaborable	`services/`, `views/`, `models/`
`reportes`	Consultas, alertas, usuarios y roles, bitácora (pantalla) y parámetros	ParametroSistema	`selectors.py`, `views.py`, `roles.py`, `signals.py`

2.5 Dependencias entre apps

- `personal` casi no depende de nadie: solo importa `licencias` para el catálogo de tipos de licencia y `reportes.auditoria` para registrar importaciones.
- `licencias` usa `personal.Designacion` y, dentro de una función, `asistencias.DiaNoLaborable` (feriados) y `asistencias.services` (marcado automático y reporte).
- `asistencias` usa `personal` y `licencias` (las referencia por texto, `"licencias.LicenciaOtorgada"`); lee `reportes.ParametroSistema` dentro de una función.
- `reportes` usa a las tres: lee todo para consultas y alertas y escucha sus señales para auditar.
- Para evitar `ImportError` circulares varios imports van **dentro de la función** (por ejemplo en `licencias/models.py` y `licencias/dias.py`).

2.6 Reglas de diseño que se repiten

Regla	Cómo se ve en el código
Capas separadas	Modelos (datos y restricciones), formularios (entrada), servicios (reglas de negocio), selectores (consultas de lectura), vistas (HTTP) y plantillas (HTML).
Todo o nada	Cada escritura va en `transaction.atomic`; el cambio y su evento de auditoría se guardan juntos o no se guarda ninguno.
No se borra, se desactiva	Casi todas las claves foráneas son `PROTECT` o `RESTRICT` (18 y 4 en los modelos); «baja» es `activo = False`.
Reglas duplicadas a propósito	Formulario (error al lado del campo) y servicio (que ninguna otra vía las saltee). Si cambia una regla hay que tocar ambos y la prueba.
Errores esperados como excepciones	`ValidationError` y `PermissionDenied`; la vista los convierte en un mensaje rojo.
Una sola fuente	«Hoy» sale de `fecha_de_hoy()`, el conteo de días hábiles de `dias.py`, los roles de `roles.py` y la configuración de `ParametroSistema`.

3. Personal (Pantalla 1)

> **En 30 segundos:** Personal guarda quién es el docente (legajo por DNI) y en qué trabaja (designaciones: cargo, materia, situación de revista, vigencia y horas por día). Cada cambio de una designación deja historial automático. Se pueden importar legajos por CSV y las designaciones que vencen se avisan y se pueden dar de baja con un comando.

3.1 Pantallas

Pantalla (URL)	Permiso	Qué hace
`/` (`inicio_personal`)	`personal.change_designacion`	Planilla de designaciones con buscador (DNI, apellido, cargo, materia, carrera) y filtro por situación; muestra las horas por día.
`/personal/legajos/`	ver o cambiar persona o designación (según la acción)	Busca un legajo y permite guardar la persona, guardar o dar de baja una designación e importar CSV. Muestra los últimos 10 errores de importación y las licencias y designaciones que vencen en 30 días.
`/personal/designaciones/nueva/`	`personal.add_designacion`	Asistente de 5 pasos; acepta `?dni=` para preseleccionar al docente.
`/personal/designaciones/<id>/editar/`	`personal.change_designacion`	El mismo asistente; con `?renovar=1` salta al paso de vigencia para cargar una nueva fecha.
`/personal/catalogos/`	según la sección	Alta, edición y baja de cargos, carreras, espacios curriculares y tipos de licencia (`?seccion=`).
`/personal/legajos/autocompletar/` y `/designaciones/autocompletar/`	login (legajos) / `change_designacion`	JSON de hasta 20 resultados para los buscadores por DNI o nombre.

3.2 Modelos

Modelo (tabla)	Campos clave	Notas
`Persona`	`dni` (PK, 7 a 10 dígitos), apellido, nombres, apellido_nombre, contacto, `activa`	El DNI no se puede cambiar (el formulario lo deshabilita al editar); «baja» es `activa = False`.
`Cargo`	`codigo` (PK, texto), descripcion, es_docente, activo	Ej.: HC (horas cátedra), COORD.
`Carrera`	nombre (único), plan_estudio, anio_plan, activa, plan_vigente, vigencia desde/hasta	Guarda el plan de estudio y su vigencia.
`EspacioCurricular`	nombre, carrera, activo	Única por (nombre, carrera): `uq_espacio_carrera_mnombre`.
`Designacion`	persona, cargo, espacio_curricular, situacion_revista (TIT/INT/SUP), desde, hasta, activo, norma_*, pof, es_suplencia, plazo_indeterminado, reemplaza	`ck_designacion_fechas`: hasta ≥ desde. Todas las FK son `PROTECT`.

Modelo (tabla)	Campos clave	Notas
`Horario`	designacion, dia (LUN..VIE), horas_catedra, hora_inicio, hora_fin	Único por (designación, día); hora_fin debe ser mayor que hora_inicio.
`HistorialDesignacion`	designacion, accion, fecha_hora, datos_anteriores y datos_nuevos (JSON)	Lo escribe `Designacion.save()` solo.
`ImportacionPersonalError`	archivo, numero_fila, dni_texto, motivo_error, fila_original	Filas rechazadas al importar CSV.

3.3 Reglas de una designación

- **Fechas:** «hasta» no puede ser anterior a «desde» (restricción de base y validación del formulario).
- **Plazo indeterminado:** solo corresponde a una suplencia y no puede tener fecha de cese.
- **Suplencia:** exige fecha de cese o plazo indeterminado. `reemplaza` apunta a la designación del titular; solo una suplencia puede reemplazar, no puede reemplazar a una designación de la misma persona y la reemplazada debe estar vigente al inicio.
- **Horas por día:** `DesignacionForm.save` recorre lunes a viernes; si hay horas hace `update_or_create` del `Horario` y si son 0 o están vacías lo borra.
- **Situación de revista:** se guarda como código `TIT`, `INT` o `SUP`; el catálogo de licencias usa los textos «Titular», «Interino», «Suplente». La validación de licencias las compara por el texto mostrado.

3.4 Historial automático

Cada vez que se guarda una designación, `Designacion.save()` trabaja dentro de una transacción:

1. Si ya existía, lee de la base los valores previos de **28 campos** (persona, cargo, materia, situación, fechas, activo, norma, POF, suplencia, plazo, observaciones, etc.).
2. Guarda y compara con los valores nuevos. Si no cambió ninguno de esos campos, no crea historial.
3. Si cambió, elige la acción: ALTA (nueva), BAJA (pasó de activa a inactiva), CAMBIO_SITUACION, TOMA_POSESION (cambió «desde») o MODIFICACION.
4. Crea un `HistorialDesignacion` con los datos anteriores y nuevos en JSON.

> **Atención: Límite:** el historial se dispara por `save()`. Un `queryset.update(...)` o un SQL directo no deja rastro, y los cambios de `Horario` no entran. Por eso el comando `cerrar_designaciones_vencidas` usa `save(update_fields=["activo"])` y no `update()`.

3.5 Asistente de 5 pasos

`nueva_designacion.html` muestra el mismo `NuevaDesignacionForm` dividido en cinco pasos con JavaScript: **1** docente (buscador por DNI o nombre), **2** cargo, materia, situación y suplencia, **3** carga horaria semanal, **4** normativa y **5** vigencia y plan de estudio. Cada paso valida lo mínimo antes de avanzar (docente elegido, cargo y situación, fecha «desde», y cese o plazo indeterminado si es suplencia). El servidor revalida todo: el JavaScript es comodidad, no seguridad.

3.6 Importación por CSV, paso a paso

1. **Archivo:** máximo 5 MB, extensión `.csv`, UTF-8, fila de encabezados y columna `dni` (los encabezados se pasan a minúsculas).
2. **Permisos:** hace falta alguno de `add_persona`, `change_persona` o `add_designacion`; además cada fila revisa el permiso exacto de lo que va a crear o modificar.
3. **Transacción por fila:** si una falla, solo esa se revierte y se guarda en `ImportacionPersonalError`.
4. **Persona:** si la fila trae datos personales crea o actualiza; si el DNI no existe y no trae datos, es error.

5. **Designación (opcional):** solo si hay `cargo_codigo`; exige que el cargo exista, que el espacio (si se indica) exista y esté activo, y `desde` en formato AAAA-MM-DD. Acepta `reemplaza_id`, `es_suplencia` y `plazo_indeterminado`.
6. **Situación de revista:** acepta TITULAR, INTERINO o SUPLENTE (o TIT, INT, SUP) sin distinguir mayúsculas.
7. **Duplicados:** rechaza una designación exacta que ya existe para esa persona.
8. **Auditoría:** al terminar registra un evento IMPORTACION con filas, importadas y rechazadas.

```
dni,apellido_nombre,cargo_codigo,espacio_curricular,situacion_revista,desde,hasta
99000001,"PEREZ, JUAN",HC,Programación I,TITULAR,2026-03-01,
```

3.7 Catálogos: desactivar o borrar

- Cargos, carreras y espacios curriculares tienen campo de estado: «eliminar» solo los **desactiva**.
- Los tipos de licencia (`LicenciaArticulo`) no tienen estado: «eliminar» intenta borrar de verdad y, si está en uso (FK `RESTRICT`), muestra «El registro está en uso y no puede eliminarse».
- La vista solo exige estar logueado y después chequea el permiso de la sección elegida (`change_cargo`, `change_carrera`, `change_espaciocurricular`, `licencias.change_licenciaarticulo`).

3.8 Comando de designaciones vencidas

`python manage.py cerrar_designaciones_vencidas` busca las designaciones activas con «hasta» anterior a hoy, las pasa a inactivas con `save(update_fields=["activo"])` (así queda el historial BAJA) e informa cuántas cerró. Está pensado para programarse una vez por día; el sistema **no** lo ejecuta solo.

3.9 Archivos

Archivo	Líneas	Qué hace
`models.py`	522	Persona, Designacion (clean y save con historial), catálogos, Horario, HistorialDesignacion, ImportacionPersonalError
`views.py`	805	Legajos, importación CSV, planilla de inicio, catálogos, alta y edición de designaciones, autocompletado
`forms.py`	407	Formularios de legajo, designación (con horas por día), nueva designación y catálogos
`admin.py` / `urls.py`	148 / 33	Administración con inlines de Horario e Historial; rutas de `/personal/`
`management/commands/cerrar_designaciones_vencidas.py`	29	Baja de designaciones vencidas con historial

4. Asistencia (Pantalla 2)

> **En 30 segundos:** Asistencia registra, día por día y por designación, una marca (Presente, Inasistencia, Licencia, Largo tratamiento o Presente retirado). El operador carga, Secretaría valida, y las jornadas y los meses se cierran. Todo cambio deja un evento con antes y después, y solo dos operadores pueden escribir a la vez.

4.1 Pantallas

Pantalla (URL)	Permiso	Qué muestra o hace
Registro diario `/asistencia/`	`view_registroasistencia`	Registros de una fecha (`?fecha=`), totales (presentes, inasistencias, sin validar, pendientes de formulario), estado del período y de la jornada y botón de cerrar jornada.
Planilla rápida `/asistencia/planilla/`	`add_registroasistencia`	Una fila por designación vigente; propone la marca y las horas; guarda todo el día de una vez.
Registrar o editar `crear/`, `editar/<pk>/`	`add_` / `change_registroasistencia`	Formulario individual con autocompletado de horas y licencias (`datos-designacion/`, JSON). Editar acepta `?volver=` (solo URLs internas).
Historial `historial/`	`view_registroasistencia`	Filtros (DNI, apellido, fechas, marca, materia, estados), 25 por página y CSV de hasta 5000 filas.
Detalle `detalle/<pk>/`	`view_registroasistencia`	Ficha del registro con sus eventos (antes → después).
Validar `validar/<pk>/`, `validar-multiples/`	`validar_registro`	Valida uno o varios por POST; redirige a `next` si es una URL interna.
Recepción `recepcion/`	`view` / `recepcionar_formulario`	Inasistencias pendientes de formulario (por defecto injustificadas y en trámite) y formulario para registrar la recepción.
Cierres `cierres/`, `confirmar/`, `<a>/<m>/reabrir/`	según la acción	Lista de cierres mensuales y diarios; confirmar cierre muestra el resumen del mes; reabrir pide motivo.
Calendario `calendario/…`	`view` / `add` / `change` / `delete_dianolaborable`	ABM de feriados por año, carga masiva y copia entre años.
Sesión `sesion/liberar/`, `sesion/renovar/`	login	Libera o renueva el lugar de edición (POST).

4.2 Modelos

Modelo (tabla)	Para qué sirve	Restricciones importantes
`MarcaAsistencia`	Catálogo cerrado de marcas; cada una define qué exige	`codigo` único; `vigentes()` devuelve solo las activas cuyo código está en `CODIGOS_MARCA`.
`RegistroAsistencia`	Una marca por designación y por día	`UNIQUE (designación, fecha)`; horas ≥ 0; si la justificación es EN_TRAMITE o JUSTIFICADA, `nro_formulario` no puede estar vacío.
`CierreDiario`	Jornada cerrada (RN-06)	`fecha` única; permiso `cerrar_jornada`.

Modelo (tabla)	Para qué sirve	Restricciones importantes
`CierreMensual`	Estado del mes: ABIERTO o CERRADO, quién cerró o reabrió, motivo y versión	`UNIQUE (año, mes)`; mes entre 1 y 12; permiso `cerrar_periodo`.
`EventoAsistencia`	Bitácora única: acción, usuario, motivo, antes y después en JSON	Inalterable por señal; también guarda eventos de licencias, usuarios y parámetros.
`SesionEdicion` / `ControlConcurrencia`	Máximo 2 operadores escribiendo a la vez	`ControlConcurrencia` es una fila fija (pk 1) usada como candado.
`DiaNoLaborable`	Feriados, asuetos e inhábiles	`fecha` única.

Las 5 marcas y lo que exige cada una

Código	Descripción	Cuenta como inasistencia	Exige observación	Exige horas > 0	Exige licencia
P	Presente	No	No	No	No
I	Inasistencia	Sí	No	Sí	No
LIC	Licencia	Sí	No	Sí	Sí
LT	Largo tratamiento	Sí	No	Sí	Sí
PR	Presente retirado	Sí	Sí	Sí	No

Se cargan por migración de datos (0003 y su corrección 0004). Las migraciones usan `update_or_create`: si se migra desde cero, los valores vuelven a los de la 0004 aunque alguien los haya editado en el admin.

4.3 Ciclo de vida de un registro

Estado	Valores	Cómo cambia
`estado_registro`	BORRADOR → VALIDADO	Nace BORRADOR. `validar_asistencia` lo pasa a VALIDADO (permiso `validar_registro`). Si se edita un validado (con motivo de 10 caracteres o más) vuelve a BORRADOR.
`estado_justificacion`	NO_APLICA, INJUSTIFICADA, EN_TRAMITE, JUSTIFICADA	Si la marca no cuenta como inasistencia es NO_APLICA. Toda inasistencia nace INJUSTIFICADA y cambia al recibir el formulario en papel.

```
INJUSTIFICADA ──► EN_TRAMITE     (se recibió el formulario)
INJUSTIFICADA ──► JUSTIFICADA    (se recibió y se aprobó)
EN_TRAMITE    ──► JUSTIFICADA
EN_TRAMITE    ──► INJUSTIFICADA  (se rechazó: exige motivo de 10 caracteres o más)
JUSTIFICADA   ──► (sin salidas: estado final)
```

Si se edita un registro y la marca deja de contar como inasistencia (por ejemplo de I a P), `_ajustar_justificacion` lo devuelve a NO_APLICA y borra número de formulario, fecha y usuario de recepción.

4.4 `guardar_asistencia`: orden de las comprobaciones

El orden importa porque define qué mensaje ve primero el usuario. Está en `services/registro.py`, decorada con `@transaction.atomic`:

1. Permiso: `add_registroasistencia` (alta) o `change_registroasistencia` (edición).
2. Sesión de edición válida (token vigente).
3. El motivo no supera los 500 caracteres.
4. El período del mes está abierto.
5. La jornada está abierta; si está cerrada exige el permiso `modificar_jornada_cerrada` y un motivo.
6. Es día laborable (no feriado ni fin de semana).
7. La designación y la persona están activas y la fecha cae dentro de la vigencia.
8. Observaciones, si la marca las exige.
9. Horas y licencia según la marca (si no pide horas, las pone en 0).
10. En edición: bloquea la fila con `select_for_update`; si estaba validada exige motivo y la pasa a borrador.
11. `full_clean()` del modelo, `_validar_licencia` y `save()`; si dos personas crean el mismo registro a la vez, el `UNIQUE` de la base se traduce a «Ya existe un registro para esa designación y fecha».
12. Crea el `EventoAsistencia` (CREAR o EDITAR) con antes y después.

4.5 Planilla rápida

- **Qué filas muestra:** designaciones activas, de personas activas y vigentes en la fecha; por defecto solo las que tienen clase ese día según el `Horario`. Si ninguna tiene horario cargado muestra todas y lo avisa.
- **Qué propone:** «Presente» en cada fila pendiente; si el docente tiene una licencia que cubre la fecha, propone LIC o LT con la licencia ya elegida; completa las horas con las del horario del día.
- **Copiar el día anterior:** el botón trae lo cargado el día hábil anterior (hasta 10 días hacia atrás) como punto de partida; no guarda nada.
- **Cuándo se bloquea:** fecha futura, día no laborable, período cerrado o jornada cerrada si el usuario no tiene `modificar_jornada_cerrada`. Con bloqueo el POST no guarda y muestra el motivo.
- **Cómo se lee lo enviado:** por cada fila con `marca-<id>` se leen `horas-<id>` (acepta coma decimal), `licencia-<id>` y `obs-<id>`; valida marca vigente, horas numéricas y que la licencia pertenezca a la designación y cubra la fecha.
- **Todo o nada:** `guardar_planilla` llama a `guardar_asistencia` por fila y junta los errores en `PlanillaConErrores`; si hay alguno se revierte la transacción completa.
- **Aviso de ciclo:** si la fecha no cae en el ciclo lectivo de los parámetros, la pantalla lo señala.

4.6 Formulario en papel y validación

- `recepcionar_formulario` (permiso `recepcionar_formulario`) exige que la marca cuente como inasistencia y que la transición esté permitida. Para EN_TRAMITE o JUSTIFICADA exige número de formulario (máx. 30) y fecha de recepción (por defecto hoy, no futura); para volver a INJUSTIFICADA borra esos datos y exige motivo.
- `validar_asistencia` falla si el registro ya estaba validado (para no dejar eventos de más) o si el período está cerrado; `validar_multiples` valida una lista e ignora los ya validados.

4.7 Cierres (RN-06)

Función	Permiso	Reglas
`cerrar_dia`	`cerrar_jornada`	No cierra una fecha futura, un día no laborable ni un período cerrado, ni dos veces. Crea `CierreDiario` y el evento CERRAR_DIA con la cantidad de registros.
`cerrar_mes`	`cerrar_periodo`	Bloquea el control de concurrencia. No deja cerrar si hay registros en BORRADOR («hay registros sin validar»). Sube la versión y registra CERRAR.

Función	Permiso	Reglas
`reabrir_mes`	`change_cierremensual`	Exige que el mes esté cerrado y motivo de 10 caracteres o más. Pasa a ABIERTO, guarda quién, cuándo y por qué, y registra REABRIR con antes y después.

Al reabrir un mes **no** se reabren los `CierreDiario` de ese mes: modificar un día cerrado sigue requiriendo `modificar_jornada_cerrada` y un motivo.

4.8 Sesión de edición: máximo dos operadores

- `MAX_OPERADORES = 2` y `MINUTOS_EXPIRACION_SESION = 15` en `services/sesiones.py`.
- `adquirir_sesion_edicion` se llama antes de cada escritura (vía `_asegurar_sesion_edicion` en las vistas). Bloquea la fila `ControlConcurrencia` (pk 1) con `select_for_update`, desactiva las sesiones vencidas, reutiliza la del mismo token y, si no hay lugar, falla con «Ya hay dos operadores activos en asistencia».
- El token (UUID) se guarda en la sesión web de Django (`asistencia_token`); el de otro usuario se rechaza.
- Las **lecturas** no necesitan sesión de edición; un tercer usuario puede mirar pero no guardar.
- Al cerrar sesión una señal libera el lugar para no bloquear a otro hasta que venza. La fila de control la crea la señal `post_migrate` de asistencias.

4.9 Calendario y días hábiles

- `motivo_dia_no_laborable(fecha)` devuelve el motivo si la fecha es feriado, asueto, inhábil o fin de semana, o `None` si es hábil. `validar_dia_laborable` lo convierte en error.
- `guardar_dia_no_laborable` y `eliminar_dia_no_laborable` no permiten tocar fechas de un período cerrado: cambiar el calendario después de cerrar alteraría qué asistencias eran válidas.
- `copiar_calendario` copia los días de un año a otro (omite el 29/02 inexistente, períodos cerrados y fechas ya cargadas; rango 2000 a 2100). `cargar_calendario_masivo` lee una fecha por línea (AAAA-MM-DD, dd/mm/aaaa o dd-mm-aaaa) con descripción opcional.
- Cargar un feriado **no** modifica asistencias ya cargadas ese día: la vista solo avisa cuántas hay.
- Para pruebas manuales, `ASISTENCIA_PERMITIR_FIN_DE_SEMANA=1` en el `.env` trata sábados y domingos como hábiles, y solo funciona con `DEBUG` activo.

4.10 Marcado automático por licencia

Al guardar o editar una licencia, `licencias/views.py` llama a `marcar_licencia` (en `asistencias/services/licencia_marcado.py`). Recorre cada fecha del período; omite fines de semana, feriados, períodos cerrados, jornadas cerradas y fechas fuera de la designación; y marca con **LT** si el tipo de licencia contiene «largo tratamiento» y **LIC** en cualquier otro caso, con las horas que la designación tiene ese día de la semana. No duplica: si el registro ya está igual, no repite el evento. Devuelve un resumen (marcados, omitidos, sin horario, advertencias) que la vista muestra como mensajes.

- Una licencia **sin fecha «hasta»** no marca ningún día y lo advierte.
- Una designación **sin horarios** no marca nada y lo advierte.
- `registros_huerfanos_de_licencia` detecta registros que quedaron vinculados a una licencia que ya no los cubre (por ejemplo, si se acortó el período); no los corrige: avisa para que se ajusten a mano.

4.11 Archivos

Archivo	Líneas	Qué hace
`views/planilla.py`	538	Planilla rápida (RF-07) y sus propuestas

Archivo	Líneas	Qué hace
`views/registro.py`	495	Registro diario, alta y edición, historial, detalle, validación, CSV y JSON de autocompletado
`views/calendario.py`, `cierres.py`, `recepcion.py`, `_comun.py`	248 / 179 / 125 / 198	Calendario, cierres, recepción y utilidades de sesión y seguridad
`forms.py`	488	Formularios de asistencia, filtros, recepción, calendario y cierres
`services/reglas.py`, `registro.py`, `recepcion.py`	269 / 208 / 231	Validaciones compartidas, guardado y recepción
`services/licencia_marcado.py`, `calendario.py`, `cierres.py`, `sesiones.py`	216 / 197 / 181 / 161	Marcado por licencia, calendario, cierres y concurrencia
`services/agenda.py`, `consultas.py`	78 / 33	Horas y licencias del día; inasistencias del período para el reporte mensual
`models/` (6 archivos), `signals.py`, `admin.py`, `urls.py`	— / 54 / 184 / 127	Modelos, señales (control de concurrencia y cierre de sesión), admin de solo lectura y rutas

5. Licencias (Pantalla 3)

> **En 30 segundos:** Licencias guarda el catálogo de artículos del Estatuto Docente (con su tope anual) y las licencias otorgadas a cada designación. Al guardar valida fechas, situación de revista, superposición y tope; marca la asistencia sola, arma el reporte mensual de horas afectadas IL-01 e IL-02 y emite las disposiciones numeradas.

5.1 Pantallas

Ruta (`/licencias/`)	Permiso	Qué hace
`nueva/`	`add_licenciaotorgada`	Alta con asistente de artículo; solo ofrece designaciones activas y vigentes en la fecha «desde».
`listado/`	`view_licenciaotorgada`	Listado de 20 por página con filtros (docente, artículo, rango de fechas); los filtros inválidos se ignoran con una advertencia.
`editar/<id>/`	`change_licenciaotorgada`	Edición; conserva los filtros del listado y siempre ofrece la designación original aunque esté dada de baja.
`reporte/<año>/<mes>/`	`view_licenciaotorgada`	Reporte mensual de horas afectadas (5.6). Año 2000 a 2100 y mes 1 a 12; fuera de rango da 404.
`generador-disposiciones/`	`change_licenciaotorgada`	GET muestra la vista previa del mes; POST emite.
`<id>/resoluciones/`, `resoluciones/<id>/descargar/`	`change_` / `view_licenciaotorgada`	Lista y adjunta PDFs; la descarga exige permiso.

5.2 Modelos

Modelo (tabla)	Campos clave	Notas
`LicenciaArticulo`	cod_licencia (único), articulo, clase, tipo_licencia, categoria_informe, sit_rev, goce_haberes, autoridad, plazo, tope_dias_anual	`categoria_informe` no tiene valor por defecto a propósito: cada artículo se clasifica en IL-01 o IL-02. Tope vacío = sin tope numérico.
`LicenciaOtorgada`	designacion, articulo, sit_rev, desde, hasta (vacío = abierta), datos de la disposición, justifica, nro_formulario, resolucion_excepcion	FK a designación y artículo con `RESTRICT`: no se borra un artículo o una designación con licencias.
`ResolucionAdjunta`	licencia, descripcion, archivo (PDF), subido_en	Se borra en cascada con la licencia; guarda en `media/resoluciones/AAAA/MM/`.
`RegistroBitacora`	accion, descripcion, fecha	Histórico: ya no se escribe; la auditoría vive en la bitácora única.

5.3 Validaciones de una licencia: `LicenciaOtorgada.clean()`

#	Regla	Mensaje (resumen)	Campo
1	Se necesitan artículo, designación y «desde»; si falta alguno no se valida nada más	—	—

#	Regla	Mensaje (resumen)	Campo
2	«Hasta» no puede ser anterior a «desde»	La fecha Hasta no puede ser anterior…	`hasta`
3	El artículo debe corresponder a la situación de revista de la designación (un suplente no usa artículos de titular)	El artículo X corresponde al personal…	`articulo`
4	Una designación no puede tener dos licencias el mismo día (una abierta se toma como sin fin)	Las fechas se superponen con otra licencia…	`desde`
5	Tope anual del artículo (RN-08), salvo resolución de excepción	Tope excedido. El límite del artículo…	`hasta`

5.4 Tope anual (RN-08), paso a paso

1. Si el artículo no tiene tope, o la licencia trae resolución de excepción (texto no vacío, ignorando espacios), no se controla nada.
2. Si no, se determinan los años que toca la licencia; si es abierta, hasta hoy. El tope es anual: cada parte se controla contra el tope de **su** año (`rango_en_anio`).
3. Por cada año: `dias_pedidos` son los días hábiles de esa parte y `dias_ya_usados` los días hábiles ya consumidos por la misma persona (mismo DNI) con el mismo artículo ese año, excluyendo la propia licencia cuando se edita.
4. Si `dias_ya_usados + dias_pedidos > tope`: error de tope excedido.

```
# licencias/dias.py - única fuente de verdad del conteo
es_dia_habil(fecha, feriados)        -> lunes a viernes y no feriado
listar_dias_habiles(desde, hasta)    -> lista de fechas hábiles
contar_dias_habiles(desde, hasta)    -> cantidad
rango_en_anio(desde, hasta, anio)    -> recorta la licencia al año; hasta=None se acota a
hoy
obtener_feriados(desde, hasta)       -> set de DiaNoLaborable (1 sola consulta)
fecha_de_hoy()                       -> timezone.localdate()
mes_vecino(anio, mes, n)             -> resuelve el cruce de año del reporte
```

> **Atención:** Los feriados del calendario de asistencias (`DiaNoLaborable`) afectan el tope: si se carga o borra un feriado, cambia cuántos días hábiles cuenta una licencia, cuánto tope queda y las alertas.

5.5 IL-01 / IL-02, disposiciones y resoluciones

Tema	Regla	Dónde y cómo
IL-01 (salud, RN-04)	Enfermedad común, corto tratamiento, largo tratamiento, enfermedad familiar	`clasificacion.py`: un diccionario (`CATEGORIAS_POR_PLANILLA`) genera los `choices` del modelo y la función de clasificación.
IL-02 (administrativas, RN-05)	Razones particulares, examen, duelo, matrimonio, compensatorio, comisión de servicio	Una categoría que no está lanza `CategoriaSinPlanillaError`: nunca se asigna IL-02 «por defecto». El reporte la muestra como «SIN CLASIFICAR».
Disposiciones (RF-16)	Solo licencias que empiezan en el mes y sin número; RN-10: solo con N° de formulario; RN-11: orden alfabético sin tildes ni mayúsculas	Numeración correlativa desde el mayor número usado para esa oficina y año; la oficina por defecto sale de `ParametroSistema`. Todo en una transacción; lo emitido no se renumera.

Tema	Regla	Dónde y cómo
Vista previa	Muestra qué números se asignarían sin guardar (GET)	`previsualizar_disposiciones_mes`; emitir es por POST. También: `manage.py emitir_disposiciones AAAA MM [--oficina X]`.
Resoluciones PDF	Extensión .pdf, máximo 5 MB y contenido que empiece con `%PDF-`	`validar_resolucion_pdf`; no existe `MEDIA_URL`: se descarga con `FileResponse` y permiso.

5.6 Reporte mensual de horas afectadas (RF-12, RF-13)

- Toma las licencias que se superponen con el mes, recorta cada una al mes y recorre día por día sumando las horas cátedra del `Horario` de la designación para ese día de la semana, **omitiendo feriados**. No filtra solo lunes a viernes porque podría existir un horario en sábado.
- Clasifica cada licencia en IL-01, IL-02 o SIN CLASIFICAR y arma una tabla por planilla con su total.
- Suma una tabla aparte con las **inasistencias del registro diario** (marcas I y PR) por designación: las licencias LIC y LT no se repiten porque ya están contadas arriba.
- Enlaces al mes anterior y siguiente con `mes_vecino`.

> **Tip:** Si el reporte muestra 0 horas para una licencia, casi siempre es porque la designación no tiene horarios cargados o porque todos los días caen en feriados.

5.7 Asistente de imputación (RF-15)

`articulos_para_asistente(situacion)` devuelve el catálogo en formato JSON (planilla de salida, tope, goce, plazo, autoridad). El formulario de nueva licencia lo usa (`datos_articulos`, plantilla `_asistente_articulo.html`) y cada opción de designación lleva `data-situacion` para filtrar los artículos que corresponden.

5.8 Archivos

Archivo	Líneas	Qué hace
`models.py`	247	LicenciaArticulo, LicenciaOtorgada (con `clean`), ResolucionAdjunta, RegistroBitacora
`views.py`	456	Alta, listado, edición, reporte mensual, disposiciones, resoluciones y marcado automático
`forms.py`	245	Formularios de licencia, filtros y resolución
`services.py`	212	Días usados, clasificación, disposiciones, asistente de artículos
`dias.py` / `clasificacion.py`	83 / 61	Conteo de días hábiles y planillas IL-01 / IL-02
`management/commands/emitir_disposiciones.py`	38	Emisión por consola

6. Administración / Reportes (Pantalla 4)

> **En 30 segundos:** Reportes reúne lo transversal. Consultas unifica licencias y ausencias con filtros y CSV; Alertas avisa topes consumidos y vencimientos; la Bitácora muestra quién cambió qué, con antes y después y sin posibilidad de editarla; y Usuarios, Roles y Parámetros configuran el sistema.

Pantalla	URL	Permiso (rol mínimo)
Consultas	`/reportes/consultas/`	`reportes.consultar_licencias` (Bedelía)
Alertas (3 pestañas)	`/reportes/alertas/`, `/alertas/designaciones/`, `/alertas/licencias-por-vencer/`	`reportes.consultar_licencias` (Bedelía)
Bitácora	`/reportes/bitacora/`	`reportes.ver_bitacora` (Secretaría)
Parámetros	`/reportes/parametros/`	`reportes.gestionar_parametros` (Subsecretaría)
Usuarios y roles	`/reportes/usuarios/`, `/nuevo/`, `/<id>/`	`reportes.gestionar_usuarios` (Subsecretaría)

6.1 Consultas (RF-17)

- `FiltroConsultaForm` valida los filtros; `buscar_consulta` devuelve una `ConsultaCombinada`: secuencia perezosa que une licencias (`buscar_licencias`) y ausencias o permisos simples (`buscar_ausencias`). Cada elemento es una `FilaConsulta` con las mismas columnas.
- Las **ausencias** son registros diarios que cuentan como inasistencia y no tienen licencia vinculada (las licencias ya se consultan como licencias). Los filtros propios de licencias (tipo, instrumento) dejan afuera las ausencias.
- **Fechas:** una licencia entra si su período se superpone con el rango; si no tiene «hasta» se considera abierta. **DNI:** por prefijo, sin puntos ni espacios. **Materia y carrera:** el formulario rechaza una materia que no pertenece a la carrera.
- Se pagina de a 25 y con `?exportar=csv` baja `consulta_licencias_y_ausencias.csv`: UTF-8 con BOM y `_texto_seguro` (apóstrofo delante de textos que empiezan con = + - @, TAB o CR) contra la inyección de fórmulas.

6.2 Alertas (RF-18)

Pestaña	Qué calcula	Filtros	Función
Topes de licencias	Docentes que usaron al menos el umbral % del tope anual de un artículo: `dias_usados_en_anio` (la misma regla de Licencias) y `round(usados*100/tope)`. «Agotado» si usados ≥ tope.	Año (por defecto el ciclo lectivo) y umbral 50, 70, 80, 90 o 100 % (defecto 80)	`calcular_alertas`
Designaciones por vencer	Designaciones activas, no indeterminadas, con «hasta» entre hoy y hoy + N días; opcionalmente las ya vencidas que siguen activas. Estados: Vencida, Vence hoy, Vence pronto (7 días o menos), Por vencer.	Anticipación 15, 30, 60 o 90 días; «incluir vencidas»	`calcular_designaciones_por_vencer`

Pestaña	Qué calcula	Filtros	Función
Licencias por vencer	Licencias cuyo «hasta» cae entre hoy y hoy + N días. Las abiertas y las terminadas no se listan.	Anticipación 15, 30, 60 o 90 días	`calcular_licencias_por_vencer`

«Hoy» sale de `fecha_de_hoy()` y los selectores aceptan un parámetro `hoy` para probar con fechas fijas. Además, la pantalla de legajos muestra un aviso con lo que vence en 30 días.

6.3 Usuarios y roles (RF-19)

- Los roles son grupos de Django con permisos acumulativos definidos en `roles.py` (tabla completa en la sección 9). `crear_roles` corre en `post_migrate` y **no toca** grupos que ya existen; `sincronizar_roles` los restablece al valor por defecto (pisa los cambios manuales).
- `UsuarioForm`: nombre y apellido obligatorios; contraseña obligatoria al crear y opcional al editar (con `validate_password` y `set_password`); el rol reemplaza solo los grupos de rol. Quien edita su propio usuario no puede cambiarse el rol ni desactivarse; un superusuario solo lo edita otro superusuario.
- El formulario **no** tiene campo `is_staff`: los usuarios creados desde la pantalla nunca entran a `/admin/`.
- Cada alta o edición queda en la bitácora; la contraseña nunca se registra, solo «restablecida».

6.4 Auditoría y bitácora (RF-20)

Origen	Acciones	Cómo se registran
Servicios de asistencias	CREAR, EDITAR, VALIDAR, RECEPCIONAR, CERRAR_DIA, CERRAR, REABRIR	Dentro de `asistencias/services/*`, en la misma transacción.
Vistas de Reportes	PARAMETROS, USUARIO_ALTA, USUARIO_EDITAR	`registrar_evento(...)` dentro de `transaction.atomic()` (también al editar parámetros desde el admin).
Señales automáticas	LICENCIA_ALTA / EDITAR / BAJA y CATALOGO_ALTA / EDITAR / BAJA	`pre_save` / `post_save` / `pre_delete` / `post_delete` sobre LicenciaOtorgada, LicenciaArticulo, Cargo, Carrera y EspacioCurricular; solo registran si hubo diferencias.
Importación de personal	IMPORTACION	`registrar_evento` desde `gestion_legajos` con filas, importadas y rechazadas.

- **Quién:** `UsuarioActualMiddleware` guarda el usuario de la petición en una variable de contexto (`contextvars`) y las señales lo leen con `obtener_usuario_actual()`. En comandos, tests o shell no hay usuario y el evento queda sin usuario.
- **Inalterable:** `impedir_edicion_de_eventos` y `impedir_borrado_de_eventos` lanzan `BitacoraInalterable`; además las FK desde la bitácora hacia registros y cierres son `PROTECT` y la del usuario también (un usuario con eventos no se borra, se desactiva).
- **Pantalla:** filtra por usuario, acción, DNI del docente, desde y hasta; 25 por página; muestra la tabla Campo / Antes / Después.

> **Atención: Límite honesto:** la inmutabilidad es a nivel aplicación. Un `QuerySet.update()` o un `UPDATE` directo en SQL no pasa por las señales; endurecerlo exigiría permisos de base de datos o triggers.

6.5 Parámetros (RF-21)

`ParametroSistema` es una tabla de una sola fila: `save()` fuerza `pk = 1` y `obtener()` hace `get_or_create(pk=1)`. Guarda código de oficina (1452 por defecto), nombre (IESETyFP) y ciclo lectivo (entre 2000 y 2100), con quién y cuándo lo modificó; cada cambio queda en la bitácora. Hoy la usan el generador de disposiciones (oficina), las alertas (ciclo por defecto) y la planilla rápida (aviso de ciclo). Los cuatro permisos propios de Reportes se declaran en `Meta.permissions` de este modelo.

6.6 Archivos

Archivo	Líneas	Qué hace
`selectors.py`	545	Eventos, licencias, ausencias, consulta combinada, alertas por tope y por vencimiento
`views.py` / `forms.py`	276 / 274	Pantallas de Reportes y sus formularios
`signals.py` / `roles.py`	178 / 101	Roles al migrar, bitácora inalterable, auditoría automática / matriz de permisos
`auditoria.py`, `exportaciones.py`	80 / 41	`registrar_evento`, acciones propias / CSV seguro
`models.py`, `middleware.py`, `usuario_actual.py`, `admin.py`	65 / 15 / 29 / 43	Parámetros y permisos; usuario de la petición; admin de parámetros con auditoría
`management/commands/sincronizar_roles.py`	14	Restablece los tres roles

7. Rúbrica de la cátedra y observaciones del docente

Esta sección contrasta el Documento Master de Requerimientos y Evaluación con el código real. El documento está escrito en términos genéricos (roles «Administrador» y «Docente», marcado de entrada y salida, aulas); el sistema resolvió el problema de otra manera. Conviene **decir primero qué sí cumple y después nombrar la diferencia**.

7.1 Criterios de aceptación de la entrega

Criterio	Cómo está en el proyecto	Dónde	Cómo decirlo
Repositorio Git/GitHub con commits descriptivos	El zip no incluye la carpeta `.git`, así que **no se puede comprobar desde el código**.	`git log --oneline` en tu máquina	Mostrá el historial en GitHub antes de la defensa y revisá que los mensajes describan el cambio.
Arquitectura MVT	Modelos en `models.py` (o `models/`), vistas en `views.py` (o `views/`), plantillas en `templates/`. Se suma una capa de servicios y selectores.	`<app>/models`, `views`, `templates/`	«Seguimos MVT y agregamos servicios para que las reglas no queden en las vistas ni en el HTML».
Modelo relacional con migraciones sin conflictos	25 migraciones en las cuatro apps (asistencias 8, licencias 10, reportes 4, personal 3); `makemigrations --check --dry-run` informa «No changes detected».	`*/migrations/`	Corré `migrate` en una base vacía y en una existente.
Autenticación, roles y permisos	Login y logout con `django.contrib.auth` (el cierre de sesión es POST con CSRF). Tres roles por grupos con permisos finos. Los usuarios los da de alta Subsecretaría; no hay auto-registro.	`config/urls.py`, `reportes/roles.py`, `reportes/views.py`	«Los roles son administrativos; el docente es un dato del sistema, no un usuario» (ver 7.3).
Interfaz HTML5 con Bootstrap responsivo	`base.html` con `viewport`, Bootstrap 5.3.3, menú lateral que se colapsa o se desliza en pantallas chicas.	`templates/base.html`, `static/css/`	Abrí una pantalla en ancho de celular durante la demo.
Documentación: requerimientos, diagramas de BD y guía de defensa	Esta guía, la documentación técnica y la Figura 3 (modelo de datos).	secciones 10 y 15	Llevá el diagrama impreso o en el celular.

7.2 Rúbrica funcional

Requisito	Cómo está en el proyecto	Estado	Dónde
Registro, inicio y cierre de sesión seguro	Login y logout de Django; alta de usuarios por Subsecretaría con validadores de contraseña y hash.	Cumple (sin auto-registro)	`reportes/forms.py` → `UsuarioForm`
RBAC: marcado restrictivo según perfil	Marcar exige `add_registroasistencia`; corregir, `change_`; validar, `validar_registro`. Sin permiso: 403.	Cumple	`asistencias/views/_comun.py`; `reportes/roles.py`
CRUD de docentes	Legajos (`Persona`) y designaciones: alta, modificación, baja lógica y consulta; importación CSV.	Cumple	`personal/views.py`
CRUD de asignaturas	`EspacioCurricular` y `Carrera` en Catálogos.	Cumple	`personal/views.py` → `catalogos_personal`

Requisito	Cómo está en el proyecto	Estado	Dónde
CRUD de aulas	**No existe un modelo de aulas.**	No implementado	—
CRUD de horarios y relación asignatura-docente-bloque	`Horario` por designación (día y horas cátedra; `hora_inicio` y `hora_fin` opcionales en el modelo). La `Designacion` une docente, materia y cargo.	Cumple (horas por día; sin aulas)	`personal/models.py`
Marcado en tiempo real de entrada y salida	El sistema registra **una marca diaria** por designación (P, I, LIC, LT, PR) cargada por el personal administrativo. No hay reloj de entrada y salida.	Distinto al enunciado	`asistencias/`
Fecha y hora tomadas del sistema, no del cliente	`creado_en` y `actualizado_en` usan `auto_now_add` y `auto_now` en el servidor; los eventos también. La **fecha del registro** la elige el operador, pero no puede ser futura, debe ser día hábil y caer dentro de la designación.	Cumple en auditoría; la fecha es elegible	`models/registro.py`, `services/reglas.py`
Justificación de inasistencias o atrasos	Inasistencias: flujo del formulario en papel (injustificada, en trámite, justificada). Los **atrasos** no tienen marca propia; lo más cercano es «Presente retirado».	Parcial	`services/recepcion.py`
Reportes por rango o por docente y exportación	Consultas con filtros y CSV, reporte mensual IL-01 / IL-02 y alertas.	Cumple	`reportes/`, `licencias/views.py`

7.3 Observaciones del docente y cómo responderlas

Observación	Qué hay en el código	Dónde	Cómo decirlo
Marcas dobles de entrada sin salida	No hay entrada y salida; el equivalente es **no duplicar el registro de un día**: restricción `UNIQUE (designacion, fecha)` en la base, y el servicio traduce el error a «Ya existe un registro para esa designación y fecha». Las filas ya cargadas de la planilla aparecen como registradas.	`models/registro.py` → `uq_asist_designacion_fecha`; `services/registro.py`	«La consistencia se garantiza en la base de datos, no solo en el formulario».
Desfase de zona horaria	`TIME_ZONE = America/Argentina/Buenos_Aires` y `USE_TZ = True`. Para «hoy» se usa `timezone.localdate()` o `fecha_de_hoy()`; en todo el código no hay un solo `datetime.now()` ni `date.today()`.	`config/settings.py`; `licencias/dias.py`; `licencias/tests/test_fecha_de_hoy.py`	«Se guarda en UTC y se muestra y compara en hora local; «hoy» tiene una única fuente».

Observación	Qué hay en el código	Dónde	Cómo decirlo
Separación estricta de roles; el rol Docente no debe tocar `/admin` ni reportes	Todas las vistas llevan `login_required` y `permission_required(..., raise_exception=True)`; sin permiso, 403. Los usuarios del sistema no son `is_staff` (el formulario no lo ofrece), así que `/admin/` no les abre. El menú oculta lo que no corresponde. Los docentes no tienen cuenta.	`_comun.py` → `vista_protegida`; `templates/base.html`; `UsuarioForm`	«Usamos permisos finos por vista en lugar de `user_passes_test`: es más preciso que preguntar por un rol».
Falta de retroalimentación tras marcar	`django.contrib.messages` en todas las vistas (éxito, error, advertencia e información), con `MESSAGE_TAGS` para que el error salga en rojo; `base.html` los muestra como alertas descartables. Al guardar una licencia se informa además cuántos días se marcaron.	`config/settings.py`; `templates/base.html`; `licencias/views.py` → `_marcar_y_avisar`	Mostralo en vivo: guardá una planilla y una licencia.

7.4 Las tres respuestas clave del documento, adaptadas a este sistema

- **Integridad del registro:** los sellos de tiempo (`creado_en`, `actualizado_en`, `creado_en` del evento) los asigna el servidor; el navegador nunca envía la hora de carga. Además el registro es único por designación y día, se valida contra el calendario y la vigencia de la designación, y cada cambio deja un evento con antes y después.
- **Acceso a rutas sensibles:** son vistas basadas en funciones, por eso se usan decoradores (`login_required`, `permission_required`) y no mixins; el control se hace en el servidor antes de renderizar, y los servicios vuelven a verificar el permiso (`_exigir_permiso`).
- **Relaciones entre modelos:** `RegistroAsistencia` apunta a `Designacion` (que une `Persona`, cargo y materia), a la marca y, opcionalmente, a la licencia. El `User` es **quien carga** (`creado_por`, `actualizado_por`), no el docente. Las FK son `PROTECT` o `RESTRICT` para no perder datos; `CASCADE` solo en horarios, resoluciones adjuntas y sesiones de edición.

> **Tip: Si te preguntan por el rol «Docente»:** el requisito se cubre sin cuentas de docente porque el sistema lo opera el personal administrativo. Una mejora natural es un rol de solo lectura para que cada docente vea su propia asistencia; hoy no existe, y conviene decirlo antes de que lo marquen.

8. Cómo se conectan los módulos (recorridos de punta a punta)

Los profesores suelen preguntar por las conexiones: «¿qué pasa cuando…?». Estos cuatro recorridos cubren las más probables.

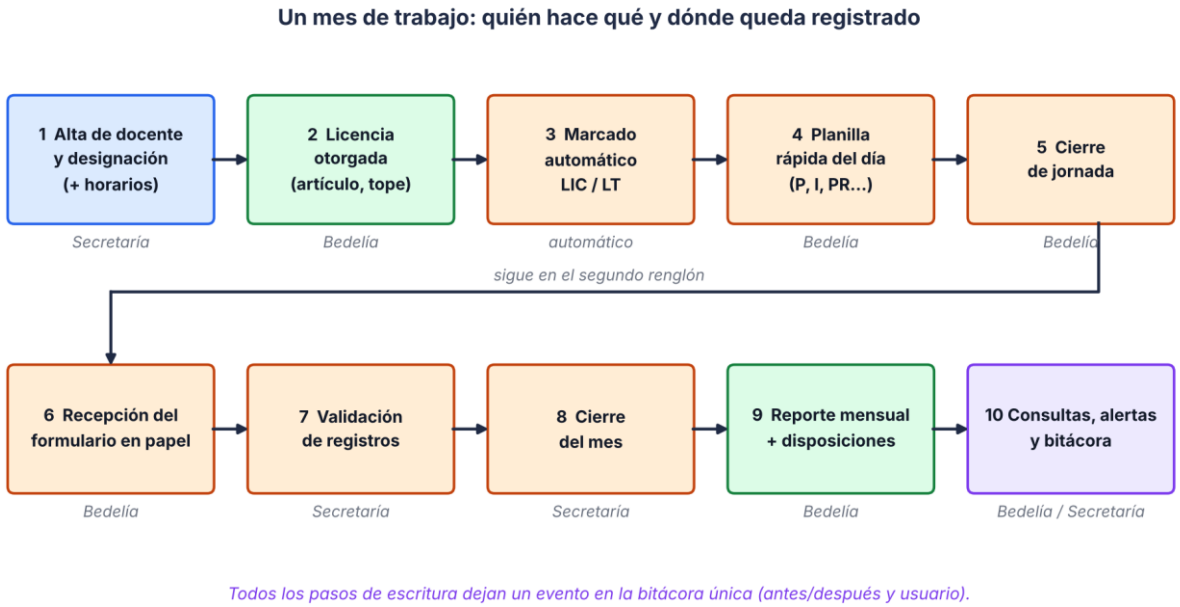


Figura 2. Un mes de trabajo: quién hace cada paso.

8.1 Se carga una licencia

1. Bedelía abre **Nueva licencia**; el asistente de artículos filtra por la situación de revista de la designación elegida (`licencias/forms.py`).
2. `LicenciaOtorgadaForm` llama a `LicenciaOtorgada.clean()`: fechas, artículo contra situación, superposición y tope anual (que consulta el calendario de `asistencias` para contar días hábiles).
3. Si pasa, `form.save()` guarda la licencia y la **señal de Reportes** crea el evento LICENCIA_ALTA con el usuario de la petición.
4. `_marcar_y_avisar` llama a `marcar_licencia` (módulo de asistencias): deja LIC o LT en cada día hábil con las horas del horario y crea un evento CREAR por día. Informa «Se marcaron N días» y advierte los omitidos o huérfanos.
5. Esos registros nacen en BORRADOR: Secretaría los valida, y ya cuentan para el reporte mensual, las alertas de tope y la consulta unificada.

8.2 Un día de clases

1. Bedelía abre la **planilla rápida**: aparece una fila por designación con clase ese día, con Presente propuesto y las horas del horario.
2. Cambia solo las excepciones (I, PR, licencias) y guarda: `guardar_planilla` valida todo y guarda todo o nada, con un evento CREAR por fila.
3. Si hubo inasistencias, cuando llega el papel se usa **Recepción de formularios** para pasar la inasistencia a en trámite o justificada.
4. Al terminar el día, **Cerrar jornada**: desde ahí corregir un registro exige permiso de Secretaría y un motivo.

8.3 Se cierra el mes

1. Secretaría valida los registros en borrador (uno por uno o varios desde el historial).

2. **Cierres → Cerrar mes**: si queda algún borrador el servicio responde «No se puede cerrar: hay registros sin validar».
3. Con el mes cerrado no se puede cargar, editar, validar ni tocar el calendario de esas fechas. Para corregir, Secretaría **reabre con motivo** (queda el evento REABRIR).
4. Con el mes cerrado Bedelía saca el **reporte mensual** (horas afectadas IL-01 / IL-02 más inasistencias) y el **generador de disposiciones**.

8.4 Un cambio y su rastro

- **Cambio de asistencia:** el servicio crea el `EventoAsistencia` con antes y después en la misma transacción.
- **Cambio de licencia o de catálogo:** `pre_save` toma la foto previa y `post_save` la compara; solo registra si hubo diferencias. El usuario sale del middleware.
- **Cambio de usuario o de parámetros:** la vista llama a `registrar_evento` (la contraseña nunca se guarda).
- **Cambio de designación:** deja `HistorialDesignacion` (no pasa por la bitácora general).
- **Lectura:** Secretaría o Subsecretaría ven todo en `/reportes/bitacora/` con filtros; cada registro de asistencia muestra además sus propios eventos en el detalle.

9. Roles y permisos: quién puede qué

Los permisos son acumulativos: Secretaría incluye todo lo de Bedelía y Subsecretaría todo lo de Secretaría. Un usuario sin rol no ve nada en el menú ni puede usar las pantallas, salvo que sea superusuario.

Capacidad (permiso)	Bedelía	Secretaría	Subsec.
Ver registro diario, historial, detalle, cierres y calendario (`view_registroasistencia`)	✓	✓	✓
Cargar y corregir asistencia, planilla rápida (`add_` y `change_registroasistencia`)	✓	✓	✓
Recepcionar formularios en papel (`recepcionar_formulario`)	✓	✓	✓
Cerrar la jornada del día (`cerrar_jornada`)	✓	✓	✓
Cargar, ver y corregir licencias; reporte mensual; disposiciones; resoluciones (`add_`, `view_`, `change_licenciaotorgada`)	✓	✓	✓
Consultas y alertas (`consultar_licencias`)	✓	✓	✓
Validar registros (`validar_registro`)	—	✓	✓
Modificar una jornada cerrada, con motivo (`modificar_jornada_cerrada`)	—	✓	✓
Cerrar y reabrir el mes (`cerrar_periodo`, `change_cierremensual`)	—	✓	✓
Calendario escolar: alta, edición, baja, carga masiva y copia (`add_`, `change_`, `delete_dianolaborable`)	—	✓	✓
Ver la bitácora (`ver_bitacora`)	—	✓	✓
Legajos y designaciones: alta, modificación, importación CSV, planilla de inicio (`add_` y `change_` persona y designacion)	—	✓	✓
Parámetros del sistema (`gestionar_parametros`)	—	—	✓
Usuarios y roles (`gestionar_usuarios`)	—	—	✓
Catálogos: cargos, carreras, espacios y tipos de licencia (`add_` y `change_`)	—	—	✓

> **Atención: Detalle para la demo:** la página de inicio `/` exige `personal.change_designacion`, que Bedelía no tiene; un usuario de Bedelía recibe 403 si abre la raíz y su pantalla natural es `/asistencia/` (a donde lo manda el login).

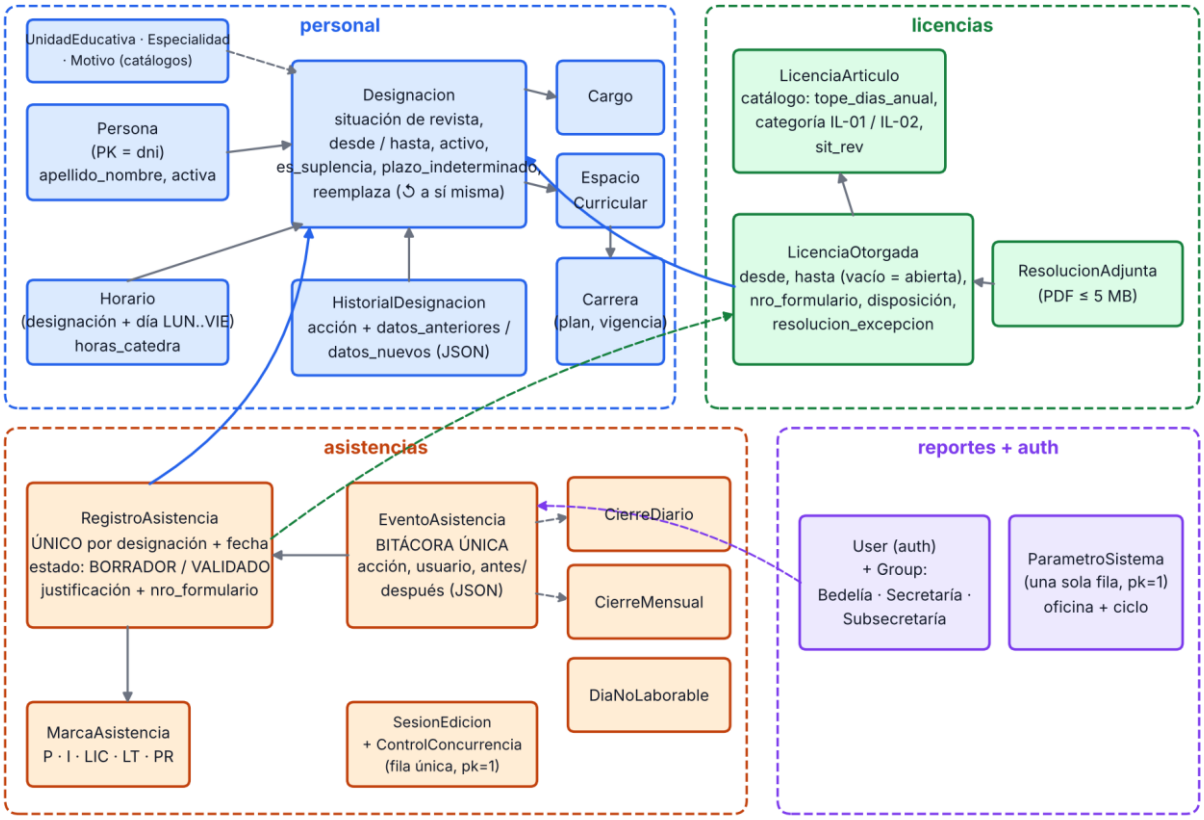
9.1 Cómo se crean y cómo se modifican los grupos

- Al correr `migrate`, la señal `crear_roles` genera los permisos de las 4 apps y llama a `crear_grupos`: crea los grupos que no existen y les asigna sus permisos; un grupo existente no se toca.
- Si se edita `roles.py`, los grupos ya existentes **no cambian solos**: hay que correr `python manage.py sincronizar_roles`.
- Las migraciones `reportes/0003_rol_lectura_asistencias` y `0004_rol_calendario_escolar` aplican a bases ya creadas los permisos que se fueron sumando.

Si agregás un permiso nuevo…	Dónde hay que tocar
Exigirlo en la vista	`@permission_required(...)` o `@vista_protegida(...)` en la vista.
Que el servicio lo valide	Llamar a `_exigir_permiso` en el servicio de asistencias.
Asignarlo a un rol	Agregarlo a `_BEDELIA`, `_SECRETARIA` o `_SUBSECRETARIA` en `roles.py` y correr `sincronizar_roles` (más una migración de datos si hay una base ya en producción).
Mostrarlo en el menú	Condición `perms.app.permiso` en `templates/base.html`.

10. Modelo de datos

Cada flecha es una clave foránea (de la tabla que la tiene hacia la tabla a la que apunta). Las líneas punteadas son relaciones opcionales.



Cada flecha es una clave foránea (de la tabla que la tiene hacia la que apunta). Las líneas punteadas son relaciones opcionales.

Figura 3. Entidades y relaciones principales, agrupadas por app.

Puntos para recordar

- **Designacion** es el centro: una persona puede tener varias, y de ella cuelgan horarios, historial, licencias y registros de asistencia.
- **LicenciaArticulo** es el catálogo (con el tope anual); **LicenciaOtorgada** es el hecho concreto, con sus fechas y su disposición.
- **RegistroAsistencia** es único por designación y fecha y puede vincularse a la licencia que lo justifica.
- **EventoAsistencia** es la bitácora: puede apuntar a un registro, a un cierre mensual o a un cierre diario, y siempre guarda el usuario.
- **ParametroSistema** es una tabla de una sola fila, sin relaciones de negocio.
- El `User` aparece como **quien hizo algo** (creado_por, actualizado_por, cerrado_por, recepcionado_por), no como el docente.

Restricciones que conviene saber nombrar

Restricción	Dónde	Qué impide
`ck_designacion_fechas`	`Designacion`	Una vigencia con «hasta» anterior a «desde».
`uq_espacio_carrera_mnombre`	`EspacioCurricular`	La misma materia repetida en una carrera.

Restricción	Dónde	Qué impide
`uq_horario_desig_dia`, `ck_horario_horas`	`Horario`	Dos horarios el mismo día; fin anterior al inicio.
`uq_asist_designacion_fecha`	`RegistroAsistencia`	Dos marcas para una designación en un día.
`ck_asist_horas_no_negativas`	`RegistroAsistencia`	Horas inasistidas negativas.
`ck_asist_formulario_requerido`	`RegistroAsistencia`	En trámite o justificada sin número de formulario.
`uq_asist_cierre_periodo`, `ck_asist_cierre_mes`	`CierreMensual`	Dos cierres del mismo mes; mes fuera de 1 a 12.
`fecha` única	`CierreDiario`, `DiaNoLaborable`	Cerrar dos veces una jornada; duplicar un feriado.
`cod_licencia` único	`LicenciaArticulo`	Dos artículos con el mismo código.

> **Nota:** Casi todas las claves foráneas son `PROTECT` o `RESTRICT`: un `ProtectedError` o `RestrictedError` al borrar significa que todavía hay registros que dependen de eso. Se **desactiva** en lugar de borrar.

11. Banco de preguntas y respuestas

Respondé con tus palabras: la idea es entender el mecanismo, no recitar. Cada pregunta indica dónde mostrarlo en el código.

A. Arquitectura y Django

P1. ¿Qué patrón usa Django y cómo se aplica acá?

Es MVT: los modelos (ORM) definen las tablas, las vistas reciben la petición y devuelven la respuesta, y las plantillas generan el HTML. Se agregó una capa de **servicios** (reglas de negocio) y de **selectores** (consultas de lectura) para que las vistas queden finas y las reglas no vivan en el HTML.

Dónde: `config/urls.py`, `<app>/views`, `asistencias/services/`, `reportes/selectors.py`

P2. ¿Cómo protegen cada pantalla?

Cada vista lleva `@login_required` y `@permission_required(permiso, raise_exception=True)`; sin permiso responde 403. En asistencias se junta todo en `vista_protegida`. Los permisos se agrupan en tres roles (grupos de Django) y los servicios vuelven a verificar el permiso.

Dónde: `asistencias/views/_comun.py`, `reportes/roles.py`

P3. ¿Qué hace `UsuarioActualMiddleware` y por qué hace falta?

Las señales (`post_save`, `post_delete`) no reciben el request, así que no saben quién hizo el cambio. El middleware guarda el usuario en una variable de contexto y las señales lo leen con `obtener_usuario_actual()`. Fuera de una petición (comandos, tests, shell) devuelve `None`.

Dónde: `reportes/middleware.py`, `usuario_actual.py`

P4. ¿Qué es un servicio y qué es un selector?

Un **servicio** es una función que cambia datos: valida permiso, sesión, período y reglas, guarda dentro de una transacción y deja el evento. Un **selector** arma una consulta de lectura sin depender del request, así la pantalla y el CSV usan la misma lógica y se prueba sin HTTP.

Dónde: `asistencias/services/`, `reportes/selectors.py`

P5. ¿Con qué base de datos corre y cómo se configura?

MariaDB/MySQL con `mysqlclient`. `settings.py` lee `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST` y `DB_PORT` del entorno (`.env`, plantilla `example.env`; el puerto por defecto es 3307). Hay un script que crea la base en utf8mb4 y otro con datos de prueba.

Dónde: `config/settings.py`, `example.env`, scripts `.sql`

P6. ¿Cómo se manejan los cambios en las tablas?

Con migraciones (`makemigrations` y `migrate`). Además `post_migrate` crea los permisos, los tres roles y la fila de control de concurrencia, así una base nueva queda lista con solo migrar.

Dónde: `*/migrations/`, `reportes/signals.py`, `asistencias/signals.py`

P7. ¿Por qué las escrituras van en `transaction.atomic`?

Para que sea todo o nada: el cambio y su evento de auditoría se guardan juntos o no se guarda ninguno. En la planilla rápida, si una fila falla se revierte el día completo.

Dónde: `asistencias/services/registro.py`

P8. ¿Qué es `select_for_update` y dónde se usa?

Bloquea filas durante la transacción para que dos peticiones no las pisen. Se usa sobre `ControlConcurrencia` (límite de operadores), sobre el registro que se edita o valida, sobre los cierres y sobre los días del calendario.

Dónde: `asistencias/services/sesiones.py`, `cierres.py`, `registro.py`

P9. ¿Cómo evitan que un enlace redirija a otro sitio?

`_url_segura` usa `url_has_allowed_host_and_scheme` con `allowed_hosts=set()`: solo acepta rutas internas en los parámetros `volver`, `desde` y `next`.

Dónde: `asistencias/views/_comun.py`

P10. ¿Qué protecciones trae Django que están activas?

CSRF (los formularios llevan `{% csrf_token %}` y el cierre de sesión es POST), escape automático de plantillas, ORM contra inyección SQL, hash de contraseñas y validadores de contraseña.

Dónde: `config/settings.py`, `templates/base.html`

B. Personal

P11. ¿Por qué el DNI es la clave primaria de `Persona`?

Porque identifica el legajo de forma natural y las importaciones y búsquedas se hacen por DNI. Es texto de 7 a 10 dígitos validado con expresión regular y **no se puede cambiar**: el formulario lo deshabilita al editar. Dar de baja es `activa = False`.

Dónde: `personal/models.py`, `forms.py` → `PersonaForm`

P12. ¿Qué es una designación y por qué es el centro del modelo?

Es la asignación de una persona a un cargo (y materia) durante un período con situación de revista. De ella cuelgan los horarios, el historial, las licencias y los registros de asistencia: la asistencia se marca por designación, no por persona.

Dónde: `personal/models.py` → `Designacion`

P13. ¿Cómo funciona el historial de designaciones?

`Designacion.save()` compara 28 campos antes y después dentro de una transacción; si alguno cambió crea un `HistorialDesignacion` con JSON anterior y nuevo y la acción (ALTA, BAJA, CAMBIO_SITUACION, TOMA_POSESION o MODIFICACION). Un `update()` masivo o un SQL directo no lo dispara.

Dónde: `personal/models.py` → `Designacion.save`

P14. ¿Cómo se modelan las suplencias?

`es_suplencia` marca la suplencia y `reemplaza` apunta a la designación del titular. La validación exige fecha de cese o plazo indeterminado, impide que alguien se reemplace a sí mismo y pide que la reemplazada esté vigente al inicio. Está en el modelo, el formulario, el asistente y la importación CSV; falta un flujo guiado de cierre al volver el titular.

Dónde: `personal/models.py` → `Designacion.clean`, `forms.py`

P15. ¿Cómo se importa personal por CSV?

`gestion_legajos` acepta un CSV de hasta 5 MB, UTF-8, con columna `dni`. Cada fila se procesa en su propia transacción con `_importar_fila_personal` y verifica los permisos de lo que crea; los errores quedan en `ImportacionPersonalError` y el resultado se audita como IMPORTACION.

Dónde: `personal/views.py` → `_procesar_csv`

P16. ¿Por qué baja lógica en vez de borrar?

Porque las claves foráneas son `PROTECT`: no se puede borrar lo que tiene licencias, asistencias o historial. Desactivar conserva la historia y la auditoría.

Dónde: `personal/models.py`, `views.py` → `catalogos_personal`

P17. ¿Para qué sirven los horarios cargados?

Dan las horas cátedra de cada día de la semana. La planilla rápida los usa para mostrar solo quienes tienen clase y proponer las horas; el marcado por licencia y el reporte mensual los usan para calcular horas afectadas.

Dónde: `asistencias/services/agenda.py`, `licencias/views.py`

P18. ¿Cómo se maneja el plan de estudios?

`Carrera` guarda `plan_estudio`, `anio_plan`, `plan_vigente` y su vigencia (desde y hasta); `EspacioCurricular` depende de la carrera con unicidad por nombre y carrera; las designaciones guardan además `pof` y `anio_pof`. Una mejora posible es una entidad de plan con historial completo.

Dónde: `personal/models.py`

P19. ¿Qué hace `cerrar_designaciones_vencidas` y por qué usa `save()`?

Da de baja las designaciones activas con «hasta» anterior a hoy. Usa `save(update_fields=["activo"])` para que se dispare el historial BAJA; un `update()` no lo dejaría. No se ejecuta solo: hay que programarlo.

Dónde: `personal/management/commands/`

C. Asistencia

P20. ¿Qué es una marca y cuántas hay?

Un código diario del catálogo cerrado: P (presente), I (inasistencia), LIC (licencia), LT (largo tratamiento) y PR (presente retirado). Cada una define si cuenta como inasistencia y si exige observación, horas o licencia vinculada.

Dónde: `asistencias/models/marcas.py`

P21. ¿Cómo se evita registrar dos veces el mismo día?

Con la restricción `UNIQUE (designacion, fecha)` en la base. El servicio traduce el error de integridad a «Ya existe un registro para esa designación y fecha». Así se cubre también la carrera entre dos operadores.

Dónde: `models/registro.py`, `services/registro.py`

P22. ¿Qué diferencia hay entre BORRADOR y VALIDADO?

Borrador es lo cargado por Bedelía; validado es lo controlado por Secretaría. Editar un validado exige un motivo de 10 caracteres o más y lo devuelve a borrador. El mes solo cierra sin borradores.

Dónde: `models/registro.py`, `services/recepcion.py`

P23. ¿Cómo funciona la justificación de inasistencias?

Toda inasistencia nace INJUSTIFICADA. Al recibir el formulario en papel pasa a EN_TRAMITE o JUSTIFICADA (con número y fecha de recepción); en trámite puede volver a injustificada con motivo; justificada es final. Las transiciones están en `TRANSICIONES_JUSTIFICACION`.

Dónde: `services/recepcion.py`

P24. ¿Qué cambia al cerrar una jornada (RN-06)?

`cerrar_dia` exige `cerrar_jornada` y sesión válida, y no cierra fechas futuras, días no laborables ni períodos cerrados. Una vez cerrada, modificar sus registros requiere `modificar_jornada_cerrada` (Secretaría o más) y un motivo.

Dónde: `services/cierres.py`, `services/reglas.py`

P25. ¿Cómo se cierra un mes y qué lo impide?

`cerrar_mes` exige `cerrar_periodo`, bloquea el control de concurrencia y rechaza si hay registros en borrador. Con el mes cerrado no se carga, edita ni valida nada ni se toca el calendario de esas fechas. `reabrir_mes` pide motivo y deja quién, cuándo y por qué.

Dónde: `services/cierres.py`

P26. ¿Por qué un máximo de dos operadores simultáneos?

Es un control de concurrencia de la carga: solo dos sesiones de edición activas a la vez, que vencen a los 15 minutos. Para que dos peticiones no se pisen se bloquea con `select_for_update` la fila única `ControlConcurrencia` antes de contar o crear sesiones.

Dónde: `services/sesiones.py`

P27. ¿Qué pasa si un operador se va sin cerrar sesión?

Su lugar vence a los 15 minutos. Si cierra sesión, una señal lo libera enseguida; también puede liberarlo con el botón de liberar sesión. Las lecturas nunca necesitan lugar.

Dónde: `asistencias/signals.py`, `views/_comun.py`

P28. ¿Cómo se tratan feriados y fines de semana?

`DiaNoLaborable` (feriado, asueto o inhábil) más los fines de semana definen los días sin marcado. `motivo_dia_no_laborable` da la razón y `validar_dia_laborable` rechaza el registro. No se puede cambiar el calendario de un mes cerrado.

Dónde: `services/reglas.py`, `services/calendario.py`

P29. ¿Cómo funciona la planilla rápida?

Arma una fila por designación vigente (por defecto solo con clase ese día), propone Presente o la licencia vigente con las horas del horario y permite copiar el día anterior. Al guardar, `guardar_planilla` valida cada fila y es todo o nada.

Dónde: `views/planilla.py`, `services/registro.py`

P30. ¿Cómo se marca la asistencia automáticamente por una licencia?

`marcar_licencia` recorre las fechas del período y deja LIC (o LT si el tipo es de largo tratamiento) en cada día hábil con las horas del horario; omite feriados, fines de semana, períodos y jornadas cerradas y avisa lo omitido. `registros_huerfanos_de_licencia` detecta lo que quedó sin cobertura al acortar una licencia.

Dónde: `asistencias/services/licencia_marcado.py`

P31. ¿Qué pasa con una licencia abierta (sin «hasta»)?

El marcado no marca ningún día y lo advierte; la superposición la trata como sin fin; el tope la cuenta hasta hoy; el reporte mensual la proyecta hasta el fin del mes; y las alertas de vencimiento no la listan porque no vence.

Dónde: `licencia_marcado.py`, `licencias/models.py`, `dias.py`

P32. ¿Por qué las reglas están en el formulario y en el servicio?

Para dar el error al lado del campo (formulario) y para que ninguna otra vía de entrada (planilla, marcado automático, otras vistas) pueda saltearlas (servicio). Si se cambia una regla hay que cambiarla en los dos lugares y en la prueba.

Dónde: `asistencias/forms.py`, `services/reglas.py`

P33. ¿Cómo manejan la zona horaria?

`TIME_ZONE = America/Argentina/Buenos_Aires` y `USE_TZ = True`: se guarda en UTC y se muestra en hora local. «Hoy» siempre sale de `timezone.localdate()` o de `fecha_de_hoy()`; no hay `datetime.now()` en el código.

Dónde: `config/settings.py`, `licencias/dias.py`

D. Licencias

P34. ¿Qué valida una licencia otorgada al guardarse?

En `clean`: «hasta» no anterior a «desde»; el artículo debe corresponder a la situación de revista de la designación; no puede superponerse con otra licencia de la misma designación; y debe respetar el tope anual del artículo, salvo resolución de excepción (RN-08).

Dónde: `licencias/models.py` → `clean`

P35. ¿Cómo se controla el tope anual?

Por cada año que toca la licencia suma los días hábiles pedidos a los ya usados por el mismo DNI con el mismo artículo (excluyendo la propia al editar). Si supera el tope, error. Una licencia que cruza de año se reparte y cada parte se controla contra su año.

Dónde: `licencias/models.py`, `services.dias_usados_en_anio`

P36. ¿Por qué solo se cuentan días hábiles y qué feriados se usan?

Es la regla del Estatuto: lunes a viernes sin feriados. Los feriados salen del calendario escolar de asistencias (`DiaNoLaborable`), por eso cargar o borrar uno cambia el consumo de tope y las alertas.

Dónde: `licencias/dias.py`

P37. ¿Qué son IL-01 e IL-02?

Las dos planillas del informe mensual: IL-01 para licencias de salud (RN-04) e IL-02 para las administrativas (RN-05). `clasificacion.py` decide según la categoría del artículo y lanza un error si no corresponde a ninguna, en vez de asignar IL-02 por defecto.

Dónde: `licencias/clasificacion.py`

P38. ¿Cómo se numeran las disposiciones internas?

`emitir_disposiciones_mes` toma las licencias que empiezan en el mes y no tienen número; solo emite las que tienen N° de formulario (RN-10), las ordena alfabéticamente por apellido (RN-11) y numera desde el último número de la oficina y el año. Lo ya emitido no se renumera.

Dónde: `licencias/services.py`

P39. ¿Qué pasa si dos personas emiten disposiciones a la vez?

Hoy podrían recibir el mismo número: la numeración lee el mayor usado y no hay bloqueo ni restricción única. Es un límite conocido; el arreglo es una restricción `UNIQUE (oficina, año, número)` o un bloqueo.

Dónde: `licencias/services.py` → `_ultimo_numero_emitido`

P40. ¿Cómo protegen los PDF de resoluciones?

Se valida extensión, tamaño (5 MB) y que el contenido empiece con `%PDF-`. No existe `MEDIA_URL`: el archivo se entrega con `FileResponse` desde una vista que exige permiso.

Dónde: `licencias/models.py`, `views.py` → `descargar_resolucion`

P41. ¿Cómo se calculan las horas afectadas del reporte mensual?

Para cada licencia que toca el mes recorre cada día y suma las horas cátedra del horario de la designación para ese día de la semana, omitiendo feriados; las agrupa en IL-01, IL-02 o sin clasificar y suma aparte las inasistencias I y PR del registro diario.

Dónde: `licencias/views.py` → `reporte_mensual_licencias`

P42. ¿Qué es el asistente de artículos?

Una ayuda del formulario de nueva licencia: `articulos_para_asistente` entrega el catálogo con planilla, tope, goce y plazo, y cada designación lleva su situación en un atributo para mostrar solo los artículos que corresponden.

Dónde: `licencias/services.py`, `forms.py`

P43. ¿Por qué `LicenciaOtorgada.sit_rev` es un texto copiado?

Es una copia informativa de la situación al momento de otorgarla. La validación compara la designación con el artículo, no con ese campo.

Dónde: `licencias/models.py`

E. Reportes, auditoría y seguridad

P44. ¿Cómo evitan la inyección de fórmulas en el CSV?

Si una celda empieza con `=`, `+`, `-`, `@`, TAB o CR, Excel la interpretaría como fórmula. `_texto_seguro` le antepone un apóstrofo. Está aplicado en las consultas de Reportes; el CSV del historial de asistencias no lo usa (ver sección 14).

Dónde: `reportes/exportaciones.py`

P45. ¿Para qué sirve el carácter invisible al inicio del CSV?

Es el BOM de UTF-8: hace que Excel abra el archivo respetando tildes y eñes.

Dónde: `reportes/views.py`, `asistencias/views/registro.py`

P46. ¿Cómo juntan licencias y ausencias en una sola consulta?

Cada origen se convierte en una `FilaConsulta` con las mismas columnas y `ConsultaCombinada` las une de forma perezosa, ordenadas por fecha, para paginar o exportar sin cargar todo en memoria.

Dónde: `reportes/selectors.py`

P47. ¿Cómo filtran por rango de fechas?

Por superposición: una licencia entra si su período toca el rango (empieza antes del «hasta» del filtro y termina después del «desde»); sin «hasta» se considera abierta.

Dónde: `reportes/selectors.py` → `buscar_licencias`

P48. ¿Cómo calculan el porcentaje del tope en las alertas?

Para cada docente y artículo con tope suman los días hábiles del año con `dias_usados_en_anio` (la misma regla de Licencias) y calculan `round(usados * 100 / tope)`. Si alcanza el umbral (80 % por defecto) aparece; si usados ≥ tope figura como agotado.

Dónde: `reportes/selectors.py` → `calcular_alertas`

P49. ¿En qué se diferencian las tres pestañas de alertas?

Topes: cuánto consumió un docente de los días anuales de un artículo. **Designaciones por vencer:** activas con «hasta» próximo o ya vencido. **Licencias por vencer:** licencias cuyo «hasta» cae en la ventana. Las dos últimas ignoran lo que no vence.

Dónde: `reportes/selectors.py`

P50. ¿Cómo garantizan que la bitácora no se pueda alterar?

Dos capas: las señales `pre_save` y `pre_delete` lanzan `BitacoraInalterable`, y las claves foráneas desde la bitácora son `PROTECT`. Es protección a nivel aplicación: un `update()` o un SQL directo no pasa por señales.

Dónde: `reportes/signals.py`, `asistencias/models/eventos.py`

P51. ¿Qué se audita y de dónde viene?

Servicios de asistencias (crear, editar, validar, recepcionar, cerrar, reabrir), vistas de Reportes (parámetros y usuarios), señales sobre licencias y catálogos, y la importación CSV. Cada evento guarda usuario, acción, motivo y fotos antes y después; en una edición solo se registra si hubo diferencias.

Dónde: `reportes/auditoria.py`, `reportes/signals.py`

P52. ¿Qué pasa si falla el guardado? ¿Queda el evento?

No. Cambio y evento van en la misma transacción: si algo se revierte, el evento también.

Dónde: `reportes/views.py`, `reportes/signals.py`

P53. ¿Cómo guardan las contraseñas?

Con `set_password` (hash) y se validan con `validate_password`. La bitácora jamás las registra: al restablecer solo anota «restablecida».

Dónde: `reportes/forms.py`, `views.usuario_editar`

P54. ¿Por qué tres roles escalonados?

Cada rol incluye los permisos del anterior. Bedelía opera el día a día, Secretaría valida, cierra y gestiona legajos, Subsecretaría administra catálogos, parámetros y usuarios. Cumple RN-06 (reabrir o modificar lo cerrado exige Secretaría o más).

Dónde: `reportes/roles.py`

P55. ¿Qué impide que alguien se quite sus permisos o toque a un superusuario?

En `UsuarioForm`, al editar el propio usuario se deshabilitan «Activo» y «Rol». En `usuario_editar`, un superusuario solo puede ser modificado por otro superusuario.

Dónde: `reportes/forms.py`, `reportes/views.py`

P56. ¿Qué pasa si cambio `roles.py`?

Los grupos que ya existen no cambian solos: `post_migrate` solo crea los que faltan. Hay que correr `python manage.py sincronizar_roles`, que los restablece (y pisa ajustes manuales).

Dónde: `reportes/roles.py`, `sincronizar_roles.py`

P57. ¿Por qué `ParametroSistema` tiene una sola fila?

Son parámetros globales: `save()` fuerza `pk = 1` y `obtener()` hace `get_or_create(pk=1)`, así todo el sistema lee la misma configuración.

Dónde: `reportes/models.py`

P58. ¿Cómo prueban el sistema?

Con `TestCase` de Django: 516 pruebas (asistencias 250, licencias 136, reportes 97, personal 33) sobre reglas, permisos, cierres, concurrencia, disposiciones, alertas y exportación. Comando: `python manage.py test`.

Dónde: `*/tests.py`, `licencias/tests/`

F. Rúbrica, decisiones y límites

P59. ¿Por qué no hay un rol «Docente»?

Porque el sistema lo opera el personal administrativo: el docente es un dato (legajo y designaciones) y la asistencia se carga sobre él. Los tres roles son administrativos y escalonados. Una mejora posible es un rol de solo lectura para que cada docente vea su propia asistencia.

Dónde: `reportes/roles.py` (sección 7)

P60. ¿Dónde está el marcado de entrada y salida?

No hay reloj de entrada y salida: se registra una marca diaria por designación (P, I, LIC, LT, PR) con las horas inasistidas. Por eso la consistencia que se cuida es «un solo registro por designación y día», no «dos entradas seguidas».

Dónde: `asistencias/models/registro.py` (sección 7)

P61. ¿Hay ABM de aulas?

No: el modelo no incluye aulas. Los horarios son por designación (día y horas cátedra, con hora de inicio y fin opcionales). Sumar aulas implicaría un modelo `Aula` relacionado con el horario.

Dónde: `personal/models.py` → `Horario`

P62. ¿Qué mejorarían con más tiempo?

Avisos proactivos (contador de alertas en el menú), paginación y exportación en las alertas por vencimiento, flujo guiado de suplencias, protección de la bitácora en la base de datos, restricción única para la numeración de disposiciones y pasar la configuración sensible al entorno.

Dónde: —

P63. ¿Qué limitaciones de seguridad tiene hoy?

`SECRET_KEY` y `DEBUG` fijos en `settings.py` (sirven para desarrollo), inmutabilidad de la bitácora solo a nivel aplicación, CSV del historial sin neutralizar fórmulas, buscador de legajos que solo exige login y permisos amplios en el script SQL. Lo que sí está activo: CSRF, hash de contraseñas, permisos por vista y escape de plantillas.

Dónde: `config/settings.py` (sección 14)

P64. ¿Cómo garantizan que el navegador no manipule la hora?

Los sellos `creado_en` y `actualizado_en` los pone el servidor y el cliente nunca envía la hora de carga. La fecha del registro la elige el operador, pero no puede ser futura, debe ser hábil y estar dentro de la designación, y queda el evento con antes y después.

Dónde: `models/registro.py`, `forms.py`

P65. ¿Por qué vistas de función y no vistas basadas en clases?

Es una decisión de estilo del equipo: cada vista es corta, delega en servicios y se protege con decoradores. Con clases se usarían `LoginRequiredMixin` y `PermissionRequiredMixin`, con el mismo efecto.

Dónde: `*/views`

12. Recetas: si te piden cambiar algo en vivo

Cada receta indica los archivos exactos y cómo comprobar que funcionó. Antes de tocar, abrí una terminal en la carpeta del proyecto (donde está `manage.py`). Cuando el cambio toca un modelo, el orden siempre es: modelo → `makemigrations` → `migrate` → formulario → plantilla → prueba.

Asistencia

R1 · Cambiar el máximo de operadores o el tiempo de expiración

Objetivo: que puedan escribir 3 operadores a la vez, o que la sesión dure 30 minutos.

Archivos: `asistencias/services/sesiones.py`

1. Cambiá `MAX_OPERADORES = 2` por 3 y/o `MINUTOS_EXPIRACION_SESION = 15` por 30.
2. Buscá el texto «dos operadores» en `adquirir_sesion_edicion` y en las pruebas de `ConcurrenciaTests`: el mensaje dice «dos» y hay que actualizarlo.
3. Corré `python manage.py test asistencias.tests.ConcurrenciaTests`.
4. Probalo con tres usuarios en tres navegadores: el tercero debería poder guardar.

R2 · Cambiar el mínimo de caracteres de un motivo

Objetivo: que reabrir un mes o modificar una jornada cerrada pida 20 caracteres en vez de 10.

Archivos: `asistencias/services/reglas.py`, `asistencias/forms.py`

1. En `reglas.py` cambiá `MOTIVO_MINIMO = 10`.
2. En `forms.py` cambiá `min_length=10` y el `help_text` de `ReaperturaMensualForm`.
3. Revisá los mensajes de las pruebas que mencionan «10 caracteres».
4. Probalo reabriendo un mes con un motivo corto.

R3 · Dejar cerrar el mes aunque haya borradores

Objetivo: cambiar la regla de cierre mensual.

Archivos: `asistencias/services/cierres.py`

1. En `cerrar_mes` buscá el bloque `pendientes = RegistroAsistencia.objects.filter(... BORRADOR ...)`.
2. Quitá el `if pendientes: raise ValidationError(...)` (o sustituilo por un aviso).
3. Actualizá la prueba que espera «hay registros sin validar» en `CierreMensualTests`.
4. Probalo cerrando un mes con un registro en borrador.

> **Atención:** Esta regla está citada en el docstring de `services/__init__.py` como ejemplo de cambio que se hace en el servicio y se prueba.

R4 · Cambiar qué exige una marca (horas, observación o licencia)

Objetivo: que «Presente retirado» no exija observación.

Archivos: admin `/admin/asistencias/marcaasistencia/`; `asistencias/models/marcas.py`

1. Entrá a `/admin/` como superusuario, abrí la marca PR y destildá «requiere observación» (el catálogo es cerrado: no se puede dar de alta ni borrar marcas).
2. El formulario y el servicio leen la bandera, así que no hay que tocar código.
3. Si hay que dejarlo permanente para otras bases, hacé una migración de datos como la `0004_corregir_marcas_asistencia`.
4. Probalo cargando un PR sin observaciones.

R5 · Agregar una marca nueva

Objetivo: sumar la marca «T» (tardanza).

Archivos: `asistencias/models/marcas.py`, `services/consultas.py`, nueva migración

1. Agregá `"T"` a `CODIGOS_MARCA`: si no, `vigentes()` la ignora.
2. Creá una migración de datos (copiá la `0003`) que cree la marca con sus banderas.
3. Definí `computa_inasistencia`: de eso dependen las ausencias en Consultas, la recepción de formularios y los totales del día.
4. Si debe contar en el reporte mensual, agregala a `CODIGOS_INASISTENCIA` de `services/consultas.py`.
5. Corré `migrate` y probala en la planilla rápida.

R6 · Agregar un feriado o cambiar qué días son hábiles

Objetivo: cargar un asueto, o considerar el sábado hábil.

Archivos: `asistencias/` (calendario), `services/reglas.py`, `licencias/dias.py`

1. **Un feriado:** entrá como Secretaría a Calendario escolar → Nuevo día (o Carga masiva, una fecha por línea). Si el mes está cerrado, no se puede.
2. **Cambiar los días hábiles:** son dos lugares con la misma idea: `DIAS_HABILES` y `DIA_HORARIO` en `reglas.py`, y `fecha.weekday() < 5` en `es_dia_habil` de `dias.py`. Hay que tocar ambos.
3. Corré `python manage.py test asistencias licencias`.

> **Atención:** Un feriado nuevo cambia cuántos días hábiles cuenta una licencia, el tope disponible y las alertas.

R7 · Agregar un campo a `RegistroAsistencia`

Objetivo: guardar el «turno» de la asistencia.

Archivos: `models/registro.py`, `forms.py`, `services/registro.py`, `services/reglas.py`, `templatetags/asistencias_extras.py`, plantillas

1. Agregá el campo al modelo y corré `makemigrations asistencias` y `migrate`.
2. Sumalo a `RegistroAsistenciaForm.Meta.fields` y a la plantilla del formulario.
3. En `guardar_asistencia` asignalo al crear y al editar.
4. Agregalo a `_datos_registro` (reglas.py) para que quede en el antes y después de la bitácora y a `CAMPO_LEGIBLE` para que se vea con nombre legible.
5. Agregá una prueba.

Licencias

R8 · Cambiar el tope de un artículo o cargar un artículo nuevo

Objetivo: que el artículo de enfermedad tenga 40 días por año.

Archivos: Catálogos → Tipos de licencia (Subsecretaría); `licencias/forms.py`

1. Entrá como Subsecretaría a Catálogos → Tipos de licencia y editá «Tope de días por año» (vacío = sin tope).
2. Para un artículo nuevo: completá código, artículo, tipo, clase, **categoría del informe** (obligatoria), situación de revista, goce, autoridad, plazo y tope.
3. Verificá en Alertas que los porcentajes cambiaron y en Nueva licencia que el tope nuevo se aplica.

R9 · Agregar una categoría a IL-01 o IL-02

Objetivo: sumar «Maternidad» a la planilla IL-01.

Archivos: `licencias/clasificacion.py`, nueva migración

1. Agregá `'Maternidad'` a la tupla de `PLANILLA_IL01` en `CATEGORIAS_POR_PLANILLA`.

2. Corré `python manage.py makemigrations licencias`: cambian los `choices` del campo (la `0007` es un ejemplo).
3. Corré `migrate`, asigná la categoría a un artículo y mirá el reporte mensual.
4. No hay que tocar el reporte: el diccionario único genera las opciones y la clasificación.

R10 · Cambiar una regla de validación de las licencias

Objetivo: permitir licencias superpuestas, o dejar de exigir que el artículo coincida con la situación.

Archivos: `licencias/models.py` → `LicenciaOtorgada.clean`; `licencias/tests/test_validaciones.py`

1. Ubicá el bloque en `clean()`: la regla de situación (comentario «M8», con `get_situacion_revista_display`) o la de superposición (consulta `superpuestas`).
2. Quitá o ajustá el bloque.
3. Actualizá o borrá la prueba correspondiente en `test_validaciones.py`.
4. Probalo cargando una licencia que antes se rechazaba.

R11 · Cambiar cómo se numeran las disposiciones

Objetivo: numerar por mes, o cambiar el texto del tipo.

Archivos: `licencias/services.py`

1. El tipo está en la constante `DISPOSICION_TIPO = 'Disposición'`.
2. La numeración sale de `_ultimo_numero_emitido(oficina, anio)`; si la cambiás, mantené coherentes `previsualizar_disposiciones_mes` y `emitir_disposiciones_mes`, que la usan igual.
3. Probalo con el generador: primero la vista previa y después emitir.
4. Si el cambio es por mes, revisá también las pruebas de `test_disposiciones.py`.

Personal

R12 · Agregar un campo a `Persona`

Objetivo: guardar el «CUIL».

Archivos: `personal/models.py`, `forms.py`, `views.py`, plantillas

1. Agregá el campo al modelo, corré `makemigrations personal` y `migrate`.
2. Sumalo a `PersonaForm.Meta.fields` (y a `widgets` si querés la clase Bootstrap).
3. Revisá `templates/personal/gestion_legajos.html`: si muestra los campos a mano, agregalo.
4. Si debe poder importarse por CSV, sumalo a la lista de campos de `_importar_fila_personal`.
5. Agregá una prueba.

R13 · Agregar una situación de revista o un campo de designación

Objetivo: un nuevo dato en la designación.

Archivos: `personal/models.py`, `forms.py`, `templates/personal/nueva_designacion.html`

1. Modelo + `makemigrations personal` + `migrate`.
2. `DesignacionForm.Meta.fields` y, si es un `Select`, su `widget`.
3. En el asistente de 5 pasos ubicá el paso donde corresponde y agregá `{{ form.campo }}`.
4. Si el dato debe quedar en el historial, sumá el campo a la tupla `campos_historial` de `Designacion.save`.

Reportes y administración

R14 · Cambiar el umbral o la ventana de las alertas

Objetivo: que arranque en 70 % y que se pueda anticipar 120 días.

Archivos: `reportes/selectors.py`, `reportes/forms.py`

1. Umbral: `UMBRAL_POR_DEFECTO = 70` en `selectors.py` (la opción 70 ya está en `FiltroAlertasForm.UMBRALES`; si usás otro valor, agregalo ahí o el formulario queda inválido).
2. Ventana: `DIAS_ANTICIPACION_POR_DEFECTO` en `selectors.py` y `(120, "120 días")` en `FiltroDesignacionesForm.ANTICIPACIONES` (la pestaña de licencias reutiliza esa lista).
3. El valor inicial siempre tiene que estar en la lista de opciones.
4. Verificá en `/reportes/alertas/`.

R15 · Cambiar qué rol puede hacer algo

Objetivo: que Bedelía también pueda ver la bitácora.

Archivos: `reportes/roles.py`

1. Agregá `("reportes", "ver_bitacora")` a `_BEDELIA` y quitalo de `_SECRETARIA` (queda por herencia, pero es más claro no repetirlo).
2. Corré `python manage.py sincronizar_roles`: los grupos existentes no se actualizan solos.
3. Entrá con un usuario de Bedelía y abrí `/reportes/bitacora/`; revisá que el menú la muestre (`perms.reportes.ver_bitacora` en `base.html`).

> **Atención:** `sincronizar_roles` pisa los ajustes de permisos hechos a mano desde el admin en esos tres grupos.

R16 · Agregar un filtro o una columna a Consultas

Objetivo: filtrar por situación de revista y exportarla.

Archivos: `reportes/forms.py`, `selectors.py`, `exportaciones.py`, `consultas.html`, `tests.py`

1. En `FiltroConsultaForm` agregá el campo; la plantilla recorre los campos y lo muestra sola.
2. En `buscar_licencias` agregá `licencias.filter(sit_rev__iexact=...)` y decidí qué pasa con las ausencias (ignorarlas o excluirlas).
3. Para una columna: atributo en `FilaConsulta`, valor en `_fila_de_licencia` y `_fila_de_ausencia`, título en `ENCABEZADO_CONSULTA` y valor en `filas_consulta` **en la misma posición**, pasando los textos por `_texto_seguro`.
4. Agregá una prueba en `reportes/tests.py`.

R17 · Agregar una acción nueva a la bitácora

Objetivo: registrar «EXPORTACION» cada vez que alguien baja un CSV.

Archivos: `reportes/auditoria.py`, `views.py`, `selectors.py`

1. En `AccionesReportes` agregá la constante y su par en `CHOICES` (el campo `accion` admite hasta 20 caracteres).
2. Llamá a `registrar_evento(request.user, AccionesReportes.EXPORTACION, motivo="consultas")` donde corresponda.
3. El filtro de acción se completa solo (`opciones_de_accion`); para el texto del objetivo, agregá la acción a `_OBJETIVO_FIJO` en `selectors.py`.

R18 · Agregar un parámetro nuevo del sistema

Objetivo: sumar «teléfono de la oficina».

Archivos: `reportes/models.py`, `forms.py`, `views.py`, `admin.py`, `parametros.html`

1. Campo nuevo en `ParametroSistema` con un default, `makemigrations reportes` y `migrate`.

2. Agregalo a `ParametroSistemaForm.Meta.fields`, a la tupla `campos` de `views.parametros` y a `CAMPOS_AUDITADOS` de `admin.py` para que quede en la bitácora.
3. Revisá `parametros.html` por si muestra los campos a mano.

R19 · Cambiar el tamaño de página de un listado

Objetivo: 50 filas por página.

Archivos: `views`

1. Historial y Recepción de asistencias: `Paginator(registros, 25)` en `_paginar` (`asistencias/views/_comun.py`).
2. Licencias: constante `LICENCIAS_POR_PAGINA = 20` en `licencias/views.py`.
3. Consultas y Bitácora: `Paginator(..., 25)` en `reportes/views.py`.

Arreglos de los puntos débiles (por si te piden corregirlos)

R20 · Neutralizar fórmulas en el CSV del historial de asistencias

Objetivo: aplicar la misma protección que Consultas.

Archivos: `asistencias/views/registro.py` → `exportar_csv`

1. Importá `_texto_seguro` desde `reportes.exportaciones`; si aparece un error de import circular, hacé el import dentro de la función `exportar_csv`.
2. En el `writer.writerow([...])` de los datos, pasá cada valor de texto por `_texto_seguro(...)`, en especial docente, cargo, materia, `nro_formulario` y usuario.
3. Agregá una prueba que cargue un `nro_formulario` que empiece con `=` y verifique el apóstrofo en el CSV.

```
fila = [r.fecha.strftime("%d/%m/%Y"), r.designacion.persona.dni, ...]
writer.writerow([_texto_seguro(v) for v in fila])
```

R21 · Exigir un permiso al buscador de legajos

Objetivo: que `/personal/legajos/autocompletar/` no sirva a un usuario sin rol.

Archivos: `personal/views.py` → `autocompletar_legajos`

1. Hoy solo tiene `@login_required`. Agregá `@permission_required("personal.change_persona", raise_exception=True)` (o una función que acepte cualquiera de los permisos de legajos, como `_puede_consultar_legajos`).
2. Verificá que la pantalla de nueva designación (que lo usa) siga funcionando para Secretaría y que un usuario sin rol reciba 403.

R22 · Crear usuarios por consola y asignar un rol

Objetivo: dar de alta un administrador o asignar un rol sin la pantalla.

Archivos: terminal

1. Superusuario: `python manage.py createsuperuser`.
2. Asignar un rol desde el shell (`python manage.py shell`).

```
from django.contrib.auth.models import User, Group
u = User.objects.get(username="maria")
u.groups.add(Group.objects.get(name="Secretaría"))
```

R23 · Un usuario recibe 403 y no debería

Objetivo: diagnosticar un permiso que falta.

Archivos: terminal, `/admin/`

1. Verificá que corrieron las migraciones: `python manage.py migrate`.

2. Restablecé los roles: `python manage.py sincronizar_roles`.
3. En `/admin/auth/group/` comprobá que el grupo tenga el permiso de la pantalla.
4. En `/admin/auth/user/` comprobá que el usuario esté en el grupo correcto.

R24 · Poner el sistema en marcha desde cero

Objetivo: preparar una computadora limpia para la demo.

Archivos: raíz del proyecto

1. Crear y activar un entorno virtual e instalar con `pip install -r requirements.txt`.
2. Crear la base y el usuario con el script SQL «crear mariadb…».
3. Copiar `example.env` a `.env` y completar `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT` (el puerto por defecto es 3307; si MariaDB escucha en 3306, hay que cambiarlo).
4. `python manage.py migrate` (crea tablas, permisos, roles, marcas y la fila de concurrencia) y `createsuperuser`.
5. Cargar `datos_prueba.sql` **una sola vez**, después de migrar; la contraseña de los usuarios de prueba está en el encabezado del archivo.
6. `python manage.py runserver` y entrar a `http://127.0.0.1:8000/`.

Si algo falla: mensaje, causa y dónde mirar

Lo que ve el usuario	Causa	Dónde mirar o cómo resolverlo
«El período está cerrado. Solicitá una reapertura autorizada.»	El mes de esa fecha está CERRADO.	Cierres: Secretaría reabre con un motivo de 10 caracteres o más.
«La jornada del … está cerrada: modificarla requiere autorización…»	Existe un `CierreDiario` de esa fecha.	Permiso `modificar_jornada_cerrada` (Secretaría o más) y motivo.
«… es feriado / fin de semana: no corresponde registrar asistencia.»	La fecha está en el calendario o cae sábado o domingo.	Calendario escolar; si fue error de carga, editar o eliminar el día (si el período está abierto).
«Ya hay dos operadores activos en asistencia.»	Hay 2 sesiones de edición activas.	Esperar 15 minutos o desactivarlas en `/admin/` (Sesiones de edición); ver `services/sesiones.py`.
«La sesión de edición expiró. Volvé a ingresar.»	Pasaron 15 minutos sin actividad o se borró la cookie.	Volver a abrir la pantalla: la sesión se readquiere.
«No existe el control de concurrencia…»	Falta la fila pk 1 de `ControlConcurrencia`.	Ejecutar `migrate` (la crea la señal `post_migrate`).
«No se puede cerrar: hay registros sin validar.»	Hay registros en BORRADOR en el mes.	Validarlos desde el Historial (filtro Estado = Borrador).
«Tope excedido. El límite del artículo…»	Días pedidos más usados superan `tope_dias_anual`.	Catálogo de artículos; cargar resolución de excepción (RN-08); revisar el calendario de feriados.
«Las fechas se superponen con otra licencia de este docente»	Otra licencia de la misma designación cubre algún día.	`LicenciaOtorgada.clean`; corregir fechas o la licencia previa.
«El artículo X corresponde al personal … y esta designación es de personal …»	La situación del artículo no coincide con la de la designación.	Elegir el artículo correcto o revisar la situación de la designación.
Aviso «No se marcaron en asistencias…» o «La licencia no tiene fecha hasta»	El marcado omitió días (período o jornada cerrados) o la licencia es abierta o la designación no tiene horarios.	Cargar horarios en la designación, completar «hasta» o corregir a mano desde la carga diaria.

Lo que ve el usuario	Causa	Dónde mirar o cómo resolverlo
Una licencia queda «omitida» al emitir disposiciones	No tiene N° de formulario (RN-10).	Editar la licencia, cargar `nro_formulario` y volver a emitir.
Los números de disposición «vuelven a 1»	Cambió el código de oficina o es otro año.	La numeración es por oficina y año; revisar Parámetros.
403 en una pantalla	El rol no tiene el permiso.	Sección 9 y receta R23.
La importación CSV rechazó filas	Datos inválidos en esas filas.	Pantalla de legajos (últimos 10 errores) o tabla `personal_importacion_error` con `motivo_error` y `fila_original`.
`BitacoraInalterable`	Se intentó editar o borrar un evento.	Es intencional; no hay forma de corregir eventos: se registra uno nuevo.
`ProtectedError` / `RestrictedError` al borrar	Hay datos que dependen del registro.	Desactivar en lugar de borrar.
Error de conexión a la base (2002, 2003, 1045)	`.env` incorrecto, MariaDB apagado o puerto distinto.	`DB_HOST`, `DB_PORT` (3307 por defecto), usuario y clave.

13. Comandos útiles

Comando	Para qué sirve
`python manage.py runserver`	Levanta el servidor de desarrollo.
`python manage.py migrate`	Aplica migraciones; además crea permisos, roles, marcas y la fila de concurrencia.
`python manage.py makemigrations <app>`	Genera migraciones después de cambiar un modelo.
`python manage.py makemigrations --check --dry-run`	Verifica que no falten migraciones (debe decir «No changes detected»).
`python manage.py check`	Revisa la configuración del proyecto.
`python manage.py test [app]`	Corre las pruebas (516 en total).
`python manage.py sincronizar_roles`	Restablece Bedelía, Secretaría y Subsecretaría a los permisos por defecto.
`python manage.py emitir_disposiciones <año> <mes> [--oficina X]`	Emite las disposiciones internas del mes (RF-16).
`python manage.py cerrar_designaciones_vencidas`	Da de baja las designaciones vencidas con historial; programarlo a diario.
`python manage.py purgar_sesiones [--dias N]`	Borra sesiones de edición inactivas viejas (por defecto, 30 días).
`python manage.py createsuperuser`	Crea un administrador.
`python manage.py collectstatic`	Reúne los archivos estáticos para producción.
`python manage.py shell`	Consola de Python con el proyecto cargado.

14. Puntos débiles conocidos y cómo responderlos

Conviene que los nombres vos antes de que te los marquen: muestra que conocés el sistema. Cada fila fue **verificada en el código**. Decí siempre qué **sí** hace el sistema antes de nombrar la limitación.

Tema	Qué pasa hoy	Cómo decirlo	Arreglo
`SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`	Están fijos en `config/settings.py` (`DEBUG = True`, hosts vacíos).	«Es configuración de desarrollo; para producción va por entorno».	Leerlos de variables de entorno, `DEBUG=False` y definir `ALLOWED_HOSTS`.
Permisos de la base	El script SQL otorga `ALL PRIVILEGES ON *.*` al usuario de la aplicación.	«Es para desarrollo local».	Limitar a la base `asistencia` y a `test_asistencia`.
Inmutabilidad de la bitácora	Se garantiza con señales y `PROTECT`; `update()` o SQL directo no pasan por señales.	«Protegida a nivel aplicación; el siguiente paso es protegerla en la base».	Permisos de BD o triggers.
Numeración de disposiciones	Se calcula leyendo el mayor número; no hay bloqueo ni `UNIQUE (oficina, año, número)`.	«Se emite de a una persona por vez en la práctica».	Restricción única o bloqueo en `emitir_disposiciones_mes`.
CSV del historial de asistencias	`exportar_csv` no usa `_texto_seguro`: un texto como `nro_formulario` que empiece con `=` llegaría como fórmula. El CSV de Consultas sí está protegido.	«La protección existe en Reportes; falta aplicarla al historial».	Receta R20.
Buscador de legajos	`/personal/legajos/autocompletar/` solo exige estar logueado: un usuario sin rol podría listar DNI y nombres.	«Está pensado para los asistentes; falta exigirle el permiso».	Receta R21.
Auditoría de personal parcial	Persona, Designación y Horario no pasan por la bitácora general; solo `HistorialDesignacion`, y solo con `save()` (el `Horario` no entra).	«Las designaciones tienen historial propio; la bitácora cubre asistencias, licencias, catálogos, usuarios y parámetros».	Sumar esos modelos a `_AUDITADOS` en `reportes/signals.py`.
Marcado automático sin sesión de edición	`marcar_licencia` no exige sesión de edición: no cuenta para el límite de dos operadores (sí audita con el usuario).	«Lo dispara el módulo de licencias, que tiene su propio permiso».	Pedir `_validar_sesion_edicion` o aceptarlo como decisión.
Licencias abiertas	No marcan días; el tope las cuenta hasta hoy pero el reporte mensual las proyecta hasta fin de mes.	«Una licencia abierta es una excepción; conviene cargar «hasta» apenas se sabe».	Unificar el criterio de proyección.

Tema	Qué pasa hoy	Cómo decirlo	Arreglo
Alertas poco proactivas	Hay que entrar a Alertas; no hay contador en el menú. La pantalla de legajos sí muestra un aviso de 30 días. Las pestañas de vencimiento no paginan ni exportan.	«Resuelve la consulta; el aviso automático es una mejora».	Contador en el menú y reutilizar la paginación y el patrón de exportación.
Suplencias	Modelo, formulario, asistente, CSV y alertas están; falta el flujo guiado (cerrar la suplencia al volver el titular, o vincularla con la licencia).	«Está resuelto lo estructural; falta el flujo automático».	Acción «finalizar suplencia» y vínculo con la licencia.
Rúbrica: entrada y salida, aulas, atrasos, rol Docente	El sistema marca una asistencia diaria por designación; no hay aulas, marca de atraso ni cuenta de docente.	«Resolvimos el control administrativo de la asistencia docente; son extensiones posibles» (ver sección 7).	Modelo `Aula`, marca de tardanza y rol de solo lectura.
Pruebas sobre otra base	La corrida de 516 tests de esta guía fue sobre SQLite; los bloqueos (`select_for_update`) no son representativos allí.	Corré la suite sobre MariaDB antes de presentar.	`python manage.py test` con tu `.env`.
Dependencia de internet	`base.html` carga Bootstrap e iconos desde un CDN.	«Sin internet la interfaz pierde estilos».	Descargar los archivos a `static/`.
Residuos	`templates/pantallas/ingreso_egreso.html` no se usa; `datos_personal.html` (en `pantallas/`) es en realidad la pantalla de inicio; `RegistroBitacora` ya no se escribe; pytest figura en requirements sin configuración; el comando de vencidas no se programa solo.	«Quedan restos del diseño inicial».	Renombrar o borrar tras verificar.
Repositorio	El zip no trae `.git`: no se pueden comprobar los commits descriptivos que pide la rúbrica.	Mostrá el historial de GitHub.	`git log --oneline` y mensajes claros.

> **Atención:** Si no sabés algo, abrí el archivo indicado en esta guía y leé la función: los nombres están pensados para que el código se explique solo.

15. Mapa de archivos

App	Líneas de código	Plantillas HTML	Qué contiene
`personal/`	1951	3	models, views, forms, admin, urls y el comando de designaciones vencidas
`licencias/`	1446	7	models, views, forms, services, dias, clasificacion, admin, urls, el comando de disposiciones y `tests/` (12 archivos)
`asistencias/`	4958	19	`models/` (6), `services/` (9), `views/` (6), forms, admin, urls, signals, `templatetags/` y el comando `purgar_sesiones`
`reportes/`	1686	9	models, views, forms, selectors, roles, signals, auditoria, exportaciones, middleware, usuario_actual, admin y el comando `sincronizar_roles`
`config/`	224	—	settings, urls, wsgi, asgi
`templates/` y `static/`	—	47 en total	`base.html`, 403, 404, 500, login, `includes/`, `pantallas/`; CSS: app, bitacora, licencias, reporte

Los conteos excluyen pruebas y migraciones (7013 líneas de pruebas en total). Las tablas por archivo están al final de las secciones 3, 4, 5 y 6.

```
Proyecto_Sistema_Asistencia-main/
├─ manage.py · requirements.txt · example.env
├─ datos_prueba.sql · crear mariadb ... .sql
├─ config/        settings.py, urls.py, wsgi.py, asgi.py
├─ personal/      legajos, designaciones, horarios, catálogos, importación CSV
├─ asistencias/   models/ services/ views/ templatetags/ management/commands/
├─ licencias/     dias.py, clasificacion.py, services.py, tests/
├─ reportes/      roles.py, signals.py, selectors.py, exportaciones.py
├─ templates/     base.html, 403/404/500, registration/, includes/, pantallas/
└─ static/        css/ (app, bitacora, licencias, reporte), img/
```

16. Guion de demo y checklist de la víspera

16.1 Guion de demo (10 a 12 minutos, por roles)

Rol	Qué mostrar y qué decir
Subsecretaría	**Parámetros** (oficina y ciclo lectivo) → **Usuarios y roles** (un solo rol por usuario; no podés quitarte el tuyo) → **Catálogos** (un artículo con tope y su categoría IL-01 / IL-02).
Secretaría	**Calendario escolar** (cargar un feriado) → **Personal**: alta de un docente y **nueva designación con el asistente de 5 pasos**, con horas por día → mostrar la importación CSV y una fila rechazada.
Bedelía	**Nueva licencia**: el asistente de artículos, un error de **tope excedido** y cómo una resolución de excepción lo permite; mostrar el aviso «Se marcaron N días en asistencias» → **Planilla rápida**: propone Presente y la licencia; provocar una fila con error para mostrar el todo o nada → **Recepción de formularios** → **Cerrar jornada**.
Secretaría	**Historial**: validar → intentar **Cerrar mes** con un borrador (error) → validar y cerrar → intentar cargar en el mes cerrado (error) → **reabrir con motivo**.

Rol	Qué mostrar y qué decir
Bedelía	**Reporte mensual** (IL-01, IL-02 y horas) → **Generador de disposiciones**: vista previa, omitidas sin N° de formulario y emitir → **Consultas** con filtros y CSV → **Alertas** (las tres pestañas).
Secretaría	**Bitácora**: abrir el evento del cambio recién hecho con antes y después; filtrar por usuario o DNI.
Cierre	Entrar como Bedelía a `/reportes/bitacora/` o `/admin/` y mostrar el **403 / 404**: la separación de roles que pide la rúbrica.

16.2 Checklist de la víspera

- Corré `python manage.py test` **con tu base MariaDB** y confirmá que no hay fallas.
- Hacé una copia de seguridad del código y de la base antes del último ensayo; `git log` y el repositorio de GitHub al día, con commits descriptivos.
- Verificá `python manage.py makemigrations --check --dry-run` («No changes detected») y `migrate` sobre una base vacía.
- Cargá `datos_prueba.sql` (una vez) y entrá con un usuario de cada rol; la contraseña está en el encabezado del archivo.
- Preparate datos para las alertas: una designación vencida, una que vence hoy, una en 5 días y una en 25; lo mismo con licencias; y un docente cerca del tope de un artículo.
- Probá en ancho de celular una pantalla (la rúbrica pide interfaz responsiva).
- Hacé un cambio de ejemplo (editá un usuario o un parámetro) y mostrá el evento en la bitácora con antes y después.
- Exportá un CSV de Consultas y abrilo en Excel: tildes bien y sin fórmulas.
- Practicá en voz alta las frases de «En 30 segundos» de cada módulo y la sección 7 (qué decir de entrada y salida, aulas y rol Docente).
- Tené abiertas en el editor las funciones de la hoja rápida: `guardar_asistencia`, `LicenciaOtorgada.clean`, `calcular_alertas`, `roles.py`, `signals.py`.
- Guardá esta guía en el celular o imprimila por si no hay conexión.